

锂离子电池快充策略对寿命衰减的影响建模与优化

摘 要

提高充电倍率可缩短补能时间,但过高电流会加速容量衰减。本文基于 MIT-Stanford 公开的 49 块 A123 18650 型磷酸铁锂电池循环老化数据,围绕两阶段快充策略 $C_1-Q_1-C_2$, 研究快充参数对寿命衰减的影响,并讨论寿命预测与策略优化问题。

针对问题一,本文对循环数据进行逐电池、逐字段的 IQR 异常检测与前后邻域均值填补,共替换 559 个异常单元格;将原始 9 种策略字符串按充电配方统一为 7 类,并以第 200 次循环的 SOH 作为观测窗口内的寿命代理指标(数据尚未覆盖 SOH = 80% 的真实寿命终止)。在 40 块满 200 圈电池中,SOH 均值为 0.9914。若仅按策略 SOH 中位数,合并后的 4.8C(80%) - 4.8C 最高,但组内离散过大、充电时间还分裂为两簇,故不以该组为典型。组内更集中的典型长寿命策略为 5C(67%) - 4C、5.3C(54%) - 4C、5.6C(36%) - 4.3C 与 5.6C(19%) - 4.6C,共同特征是 $C_1 > C_2$,充电时间约 10.04 min;典型短寿命策略为 3.7C(31%) - 5.9C,特征为 $C_1 < C_2$,在相近充电时间下 SOH 明显更低。结果表明:高 SOC 区间维持高倍率更伤寿命;提高电流时适当降低切换 SOC Q_1 有助于在几乎不增加充电时间的前提下保持健康状态。温度、内阻与初始容量不能单独解释这一长短寿差异。

针对问题二,本文以与赛题同源、且测试标签隔离的公开 EOL 为寿命响应,在同一实验结构的 30 块去重非测试电池中比较六种策略。策略间寿命分布总体存在差异(Kruskal-Wallis: $H = 11.614, p = 0.0405$),但 Holm 校正后的两两比较均未显著。固定 SOC 窗口的两阶段电流暴露模型可解释部分策略差异,却在留一策略时出现最高 95.8% 的误差,故仅作机制线索,不能外推为单个充电参数的因果定律。

针对问题三,本文在 40 块完整电池上按电池 5 折验证“前 150 圈预测后 50 圈”的任务。最近 50 圈线性尾段外推的 RMSE 为 0.000308,优于小型 MLP 的 0.000464,故用于 9 块测试电池的 151-200 圈 SOH 预测。仅凭早期平稳 SOH 直接外推至 80% 会产生不稳定的超长寿命,故额外引入公开同源、但与测试标签隔离的 34 块非测试电池 EOL 进行标定;岭回归的留一验证中位绝对百分误差为 11.75%,并给出带数百圈不确定性的 EOL 风险估计。

针对问题四,拟建立充电时间模型与寿命衰减模型,在已有实验策略及其合理邻域内进行充电时间-寿命的多目标权衡,给出推荐策略并讨论外推风险。(本节定量结果待补全)

关键词: 锂离子电池; 快充策略; SOH; 循环寿命; IQR; 寿命预测; 多目标优化

1 问题重述

随着新能源汽车与储能系统对补能效率的要求提高，锂离子电池快充成为缩短充电时间的主要手段。然而较高充电电流会增强极化、加剧热效应并加速副反应，从而加快容量衰减、缩短循环寿命 [1–3]。电池健康状态定义为

$$\text{SOH} = \frac{Q_{\text{current}}}{Q_{\text{initial}}}, \quad (1)$$

本题以 $\text{SOH} = 80\%$ 作为寿命终止 (EOL) 判定阈值。

本题充电过程采用两阶段快充：以倍率 C_1 从 0% SOC 充至切换点 Q_1 ，再以倍率 C_2 从 Q_1 充至 80% SOC，随后以 1C 恒流–恒压 (CC-CV) 补至满电，至电流降至 $C/50$ 结束。各电池放电条件一致，主要差异在充电策略。数据来自 MIT–Stanford 公开循环老化实验 [1, 4]，包含 49 块额定容量 1.1 Ah 的 A123 18650 磷酸铁锂电池。

需要完成的研究任务为：

- 问题 1.** 整理数据，提取策略、充电时间、SOH 变化与循环寿命信息，分析寿命分布并解释典型长/短寿命策略差异。
- 问题 2.** 分析策略参数 C_1, Q_1, C_2 对循环寿命的影响及其统计显著性与可能机制。
- 问题 3.** 基于截至第 150 次循环的数据预测第 151–200 次循环 SOH，并外推 EOL 循环寿命。
- 问题 4.** 兼顾充电时间与寿命衰减，在实验策略及其合理邻域内优化快充策略。

2 问题分析与总体思路

问题一属于探索性数据分析：在尚未达到真实 EOL 的观测窗口内，需要定义可比较的寿命代理指标，并完成清洗、可视化与典型策略提取。问题二强调参数非正交，不宜按完全析因实验解释因果关系。问题三是早期循环预测与外推。问题四是充电时间与衰减率的多目标决策。四问递进关系为：数据整理 → 影响分析 → 寿命预测 → 策略优化。

问题三先回答可由隐藏第 151–200 圈评分的短期预测，再在明确数据边界的前提下估计 EOL；问题二以独立的非测试公开 EOL 样本解释策略关联。问题四仍待完成。

3 符号说明与基本假设

3.1 主要符号

3.2 基本假设

1. 各电池放电协议一致，寿命差异主要由充电策略及电池个体差异引起。

表 1 主要符号说明

符号	含义
SOH	电池健康状态, 见式 (1)
C_1, C_2	第一、二阶段充电倍率 (C-rate)
Q_1	阶段切换 SOC (%)
N	循环次数
N_{EOL}	SOH = 80% 对应的循环寿命
SOH_{200}	第 200 次循环的 SOH (寿命代理)
T_{charge}	平均充电时间 (min)

2. 在 200 次循环窗口内未达到 SOH = 80%, 故问题一以 SOH_{200} 作为寿命代理指标, 并在文中明确其与真实循环寿命的区别。
3. 问题一按充电配方将原始策略字符串统一为 7 类 (去掉 NEWSTRUCTURE 后缀并合并同名记录); 合并组若组内离散明显, 不作为典型策略。
4. IQR 筛出的离群点视为统计异常而非已证实的传感器故障, 清洗仅用于本题指定分析。
5. 问题四若扩展连续参数, 取值限制在已有实验数据的凸包或合理邻域内, 避免无约束外推。

4 问题一：数据整理与快充策略对寿命影响的初步分析

4.1 数据来源与结构

本题使用赛题提供的锂离子电池循环老化数据 [1, 4] (MIT/Stanford 公开数据集)。battery_summary.csv 记录 49 块电池的策略参数 C_1, Q_1, C_2 、初始容量、平均充电时间、平均内阻、平均温度及问题三测试标记 prediction_test; cycle_train.csv 记录逐循环的容量、SOH、平滑 SOH、充电时间、内阻与平均温度, 共 9350 行。非测试电池记录至第 200 次循环, 9 块测试电池仅至第 150 次循环。输入质量检查表明: battery_id+cycle 无重复; 循环表无缺失; 汇总表有 3 个缺失单元格, 其中部分电池 C_1 为结构性缺失, 予以保留且不参与循环数据清洗。最大循环数分布为 {150 : 9, 200 : 40}。

4.2 寿命指标定义

题设循环寿命为 SOH 降至 80% 时的循环次数 N_{EOL} 。本数据集在 200 次循环内 SOH_{200} 范围为 0.9497 ~ 1.0002, 均值 0.9914, 尚未覆盖真实寿命终止。因此问题一采用

$$\text{LifeProxy}_i = \text{SOH}_i(N = 200) \quad (2)$$

作为观测窗口内的使用寿命代理：该值越高，表示前 200 圈健康保持越好。直方图与箱线图参考线取样本上、下四分位数，用于刻画分布位置，而不再直接等同于“寿命终止阈值”。若需与问题二、三衔接，可进一步计算衰减率

$$\text{FadeRate}_i = \frac{\text{SOH}_i(1) - \text{SOH}_i(N)}{N - 1}. \quad (3)$$

4.3 IQR 清洗

对每块电池、每个测量字段（capacity、SOH、SOH_smooth、chargetime、IR、Tavg）分别计算下、上四分位数 Q_1, Q_3 及四分位距 $\text{IQR} = Q_3 - Q_1$ 。将

$$x \notin [Q_1 - 1.5\text{IQR}, Q_3 + 1.5\text{IQR}]$$

的观测标为异常，并用同一电池、同一字段中距离最近的前后正常值均值替换；若仅有单侧邻域则用该侧值。共替换 559 个单元格，分字段数量为：capacity 36、SOH 36、SOH_smooth 17、IR 142、Tavg 142、chargetime 186。原始 data/ 文件未改写，清洗结果保存为 cycle_train_cleaned.csv。

4.4 快充策略统一命名

原始记录中共出现 9 种策略字符串。第三次迭代按充电配方将同名或仅差后缀的策略合并，并去掉 _NEWSTRUCTURE 后缀，得到 7 种统一策略，映射关系见表 2。验收确认：原始 9 种减为 7 种，旧名称无残留，49 块电池的 charge_policy.csv 均采用统一名称。

表 2 原始策略名称到统一策略名称的映射

原始名称	统一名称
3_6C-80PER_3_6C、80PER_3_6C	3_6C-80PER_3_6C
4_8C_80PER_4_8C、4_8C_80PER_4_8C_NEWSTRUCTURE	4_8C_80PER_4_8C
5C_67PER_4C_NEWSTRUCTURE	5C_67PER_4C
5_3C_54PER_4C_NEWSTRUCTURE	5_3C_54PER_4C
5_6C_19PER_4_6C_NEWSTRUCTURE	5_6C_19PER_4_6C
5_6C_36PER_4_3C_NEWSTRUCTURE	5_6C_36PER_4_3C
3_7C_31PER_5_9C_NEWSTRUCTURE	3_7C_31PER_5_9C

统一后的 7 种策略对应两阶段参数见表 3。其中 3_6C-80PER_3_6C 组部分电池原始 C_1 缺失，表中记为“3.6/缺失”。寿命统计分析仅纳入最大循环数恰为 200 的 40 块电池，排除 9 块问题三测试电池。

4.5 SOH 变化曲线

全部曲线均采用清洗后的 SOH_smooth。图 1 给出 1 号电池示例；49 块电池的单体曲线见支撑材料 A/问题一/SOH/SOH_N.png。图 2 将全部电池叠绘于同一坐标系，颜色

表 3 统一策略的充电参数及过程量（200 圈样本）

统一策略	C_1	Q_1	C_2	n	$\overline{IR}$	$\overline{T}/^{\circ}\text{C}$	$\overline{T}_{\text{ch}}/\text{min}$
3_6C-80PER_3_6C	3.6/缺失	80	3.6	4	0.0170	31.84	13.382
4_8C_80PER_4_8C	4.8	80	4.8	7	0.0159	32.43	10.760
5C_67PER_4C	5.0	67	4.0	6	0.0155	34.04	10.043
5_3C_54PER_4C	5.3	54	4.0	7	0.0156	34.36	10.043
5_6C_19PER_4_6C	5.6	19	4.6	7	0.0152	35.41	10.043
5_6C_36PER_4_3C	5.6	36	4.3	7	0.0157	35.21	10.043
3_7C_31PER_5_9C	3.7	31	5.9	2	0.0139	35.68	10.134

注： n 为满 200 圈电池数；充电时间、内阻与温度为各电池 1-200 圈均值后再按策略平均。

由蓝到红随 battery_id 递增。可见多数电池在 200 圈内衰减缓慢，少数曲线明显下探，后文箱线图表明这些快速衰减样本主要集中于 3_7C_31PER_5_9C 以及合并后离散较大的两组。

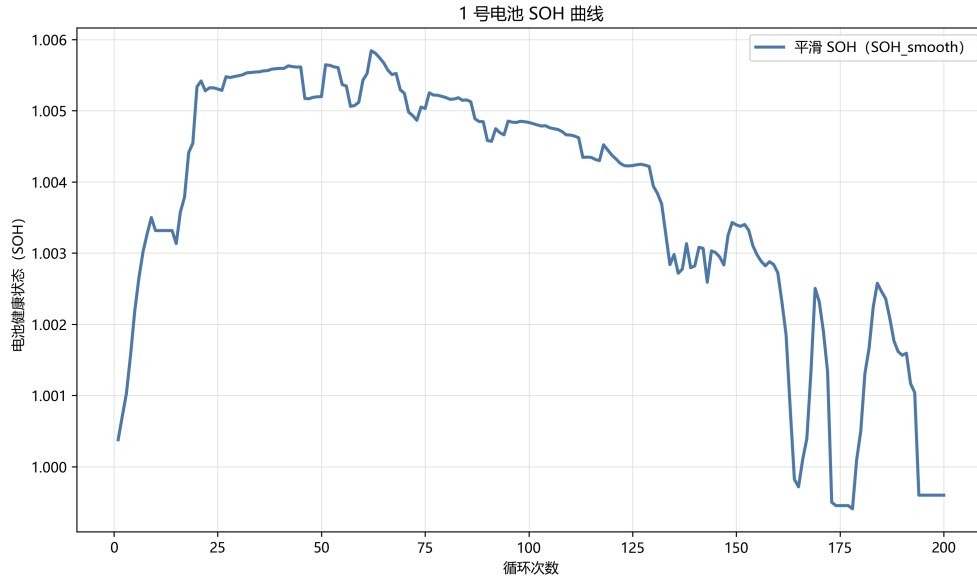


图 1 1 号电池平滑 SOH 曲线

4.6 寿命分布统计分析

对 40 块满 200 圈电池的 SOH_{200} 绘制直方图，见图 3。样本上、下四分位

$$Q_3 = 0.996103, \quad Q_1 = 0.993433$$

分别以红色、蓝色虚线标出，用作分布位置参考。据此可将单体分为高于 Q_3 的 10 块、介于 Q_1 与 Q_3 的 20 块、低于 Q_1 的 10 块。需要强调：该划分针对的是电池个体，且全样本 SOH_{200} 高度集中在 0.99 附近，多数 $C_1 > C_2$ 策略的电池落在中间箱体。因此不能把“策略中位数是否越过 Q_3 ”直接等同于“典型长寿命策略”；后文改按组内集中程度与 C_1, C_2 形态提取典型策略。

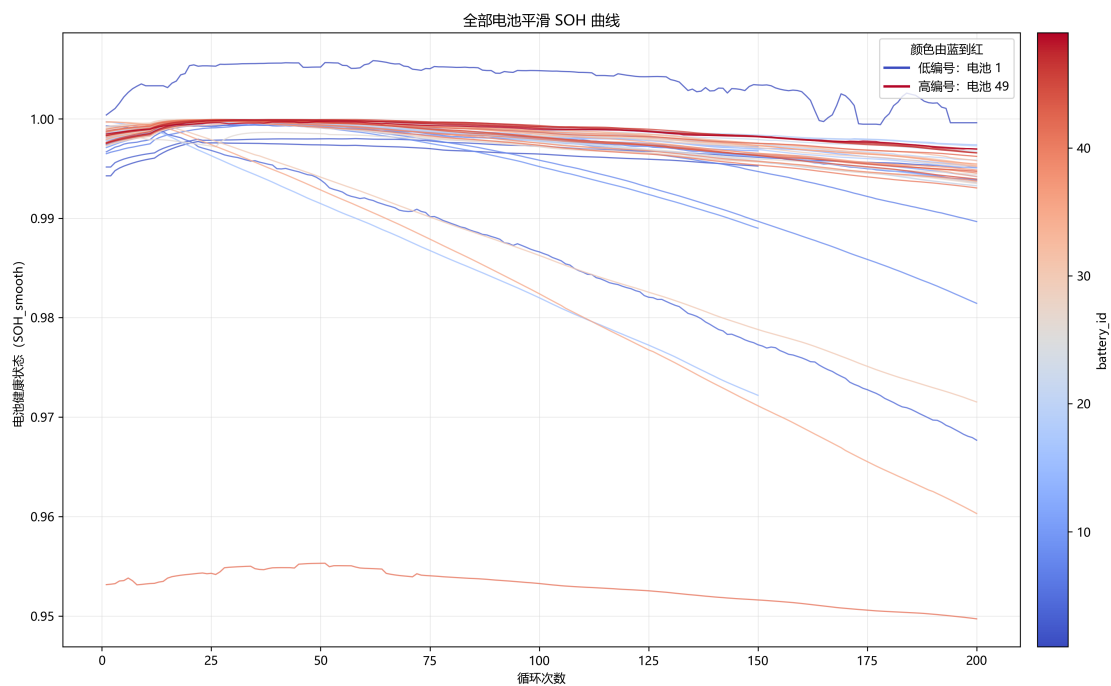


图 2 全部 49 块电池的平滑 SOH 曲线（颜色由蓝到红对应电池编号增大）

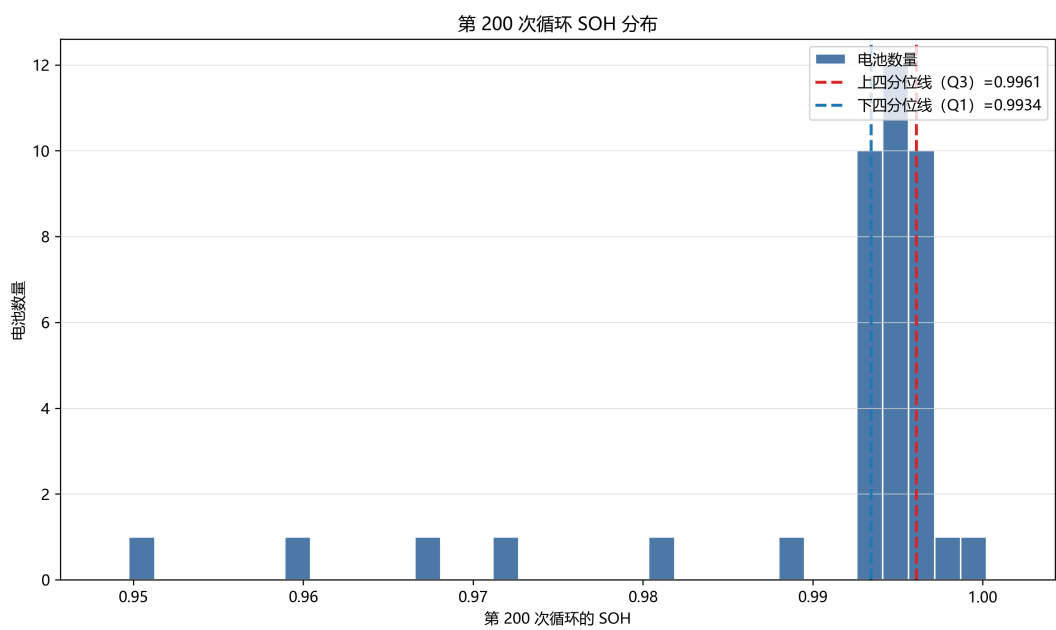


图 3 第 200 次循环 SOH 分布及上、下四分位参考线

SOH₂₀₀ 与平均充电时间的散点图见图 4。四类 $C_1 > C_2$ 的分段快充策略充电时间均集中在约 10.04 min，SOH 也较高；3_7C_31PER_5_9C 充电时间相近（约 10.13 min）但 SOH 明显更低，说明在相近补能时间下，策略形态仍可造成寿命差异。3_6C-80PER_3_6C 充电时间最长（约 13.38 min），但 SOH 并不一律最高。4_8C_80PER_4_8C 的充电时间分裂为约 10.06 min（电池 7、8）与约 11.04 min（电池 17、20、41、46、49）两簇，后者中位数虽高，却包含 SOH₂₀₀ $\approx$ 0.950 的显著偏低个体（电池 41）。

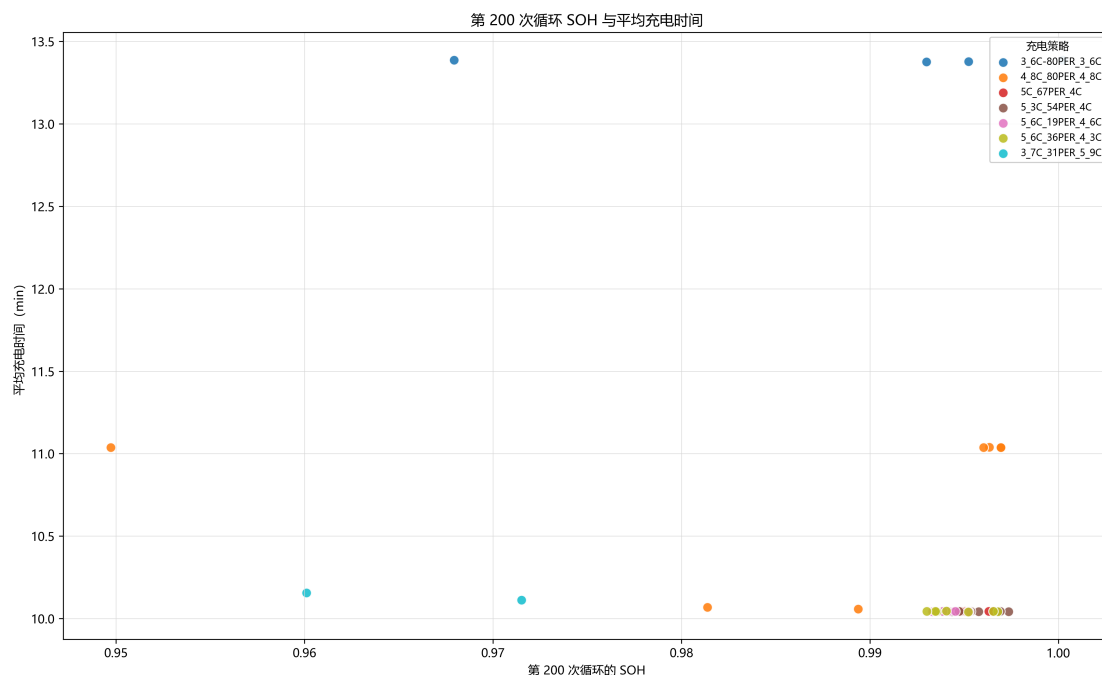


图 4 第 200 次循环 SOH 与平均充电时间散点图（按统一策略着色）

不同统一策略下 SOH₂₀₀ 的箱线图见图 5，策略汇总见表 4。

表 4 统一策略第 200 次循环 SOH（均值/中位数）与充电时间汇总

统一策略	电池记号	n	SOH 平均	中位数	SOH 范围	充电时间/min
5C_67PER_4C	18,29,31,37,42,47	6	0.9955	0.9952	0.9948–0.9966	10.042–10.044
5_3C_54PER_4C	19,26,32,36,38,43,48	7	0.9955	0.9954	0.9938–0.9974	10.042–10.044
5_6C_36PER_4_3C	13,15,22,28,34,40,45	7	0.9947	0.9941	0.9930–0.9968	10.041–10.045
5_6C_19PER_4_6C	12,21,23,27,33,39,44	7	0.9940	0.9939	0.9933–0.9953	10.042–10.044
3_6C-80PER_3_6C	1,3,4,6	4	0.9891	0.9941	0.9679–1.0002	13.376–13.388
4_8C_80PER_4_8C	7,8,17,20,41,46,49	7	0.9867	0.9960	0.9497–0.9970	10.058–11.038
3_7C_31PER_5_9C	30,35	2	0.9658	0.9658	0.9601–0.9715	10.111–10.156

注：按 SOH 平均值降序。若改按中位数，则 4_8C_80PER_4_8C 最高（流水线据此标记“寿命最长”），但其均值被低尾个体拉低。测试电池未纳入。完整参数见 policy.csv。

4.7 典型长寿命与短寿命策略

箱线图显示，3_6C-80PER_3_6C 与 4_8C_80PER_4_8C 的 SOH₂₀₀ 在组内大幅波动，不宜作为“典型”代表。前者由 80PER_3_6C 与 3_6C-80PER_3_6C 合并，部分电池 C_1

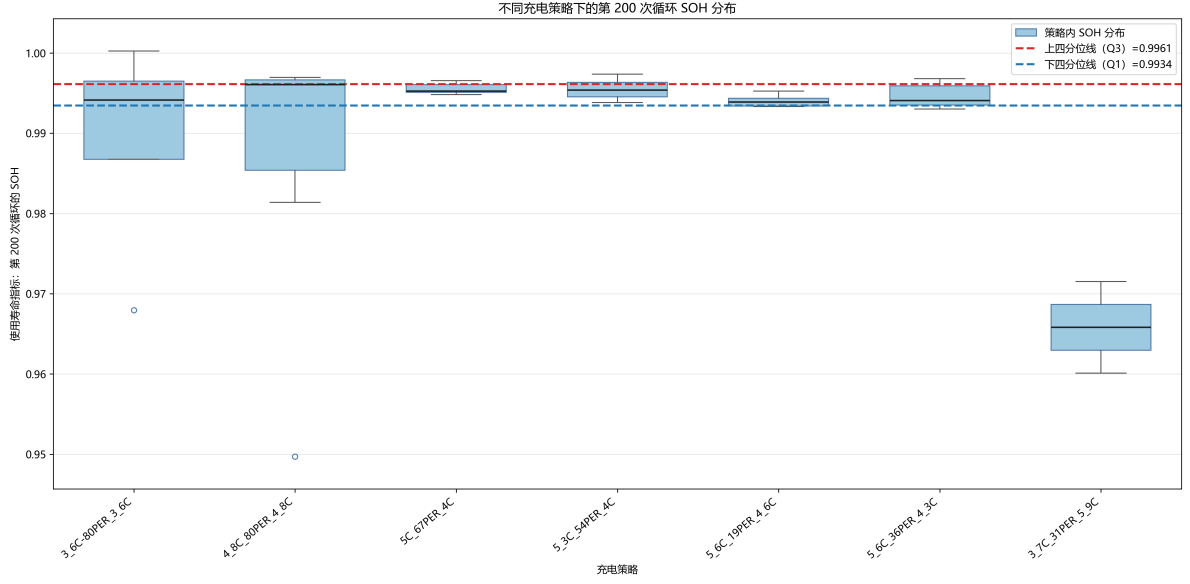


图 5 不同统一充电策略下第 200 次循环 SOH 箱线图

缺失，既含全样本最高个体（电池 1， $\text{SOH}_{200} \approx 1.000$ ），也含明显偏低个体（电池 4， ≈ 0.968 ）；后者合并带/不带 NEWSTRUCTURE 的记录，充电时间分为约 10.06 min 与 11.04 min 两簇。流水线若只按策略 SOH 中位数会把 4_8C_80PER_4_8C 标为寿命最长，但这恰好说明：高中位数可以被少数高健康样本抬起，而不能代表该配方的稳健水平。因此典型策略改从组内更集中的分段快充中提取，与 A/问题一/总结.md 的判定一致。

典型长寿命策略（组内 SOH 高且离散小，充电时间约 10.04 min）：

- 5C_67PER_4C: $C_1 = 5.0 > C_2 = 4.0$, $Q_1 = 67\%$, SOH 平均 0.9955;
- 5_3C_54PER_4C: $C_1 = 5.3 > C_2 = 4.0$, $Q_1 = 54\%$, SOH 平均 0.9955;
- 5_6C_36PER_4_3C: $C_1 = 5.6 > C_2 = 4.3$, $Q_1 = 36\%$, SOH 平均 0.9947;
- 5_6C_19PER_4_6C: $C_1 = 5.6 > C_2 = 4.6$, $Q_1 = 19\%$, SOH 平均 0.9940。

典型短寿命策略：3_7C_31PER_5_9C, $C_1 = 3.7 < C_2 = 5.9$, $Q_1 = 31\%$, SOH 平均仅 0.9658，两块电池均明显低于下四分位线。其平均充电时间约 10.13 min，与长寿命组几乎同量级，因此衰减差异主要来自策略形态而非“充得更快”。

4.8 长、短寿命策略差异的解释

结合两阶段快充的物理图像 [2,3,5,6]，可将问题一的关键差异概括如下。

1. **电流分配**: $C_1 > C_2$ 优于 $C_1 < C_2$ 。典型长寿命策略均满足前段倍率高于后段，即在低 SOC 区间采用较大电流、在中高 SOC 区间回落到较低电流。短寿命策略则相反：后段 $C_2 = 5.9C$ 为全样本最高。高 SOC 时电解液中锂离子浓度梯度更大、石墨负极嵌锂位点更拥挤，过电位升高后更易诱发锂沉积与不可逆容量损失；同时极化与产热叠加，加速 SEI 生长 [2,3]。LFP 体系虽相对耐过充，但仍可通过活性

锂损失 (LLI) 表现为早期 SOH 下降 [5]。多段恒流对照亦表明, 把大电流尽量安排在低 SOC 段更有利于寿命 [6]。

2. **恒流高 SOC 充电: 提高 $C_1 = C_2$ 时均值与下尾变差, 但中位数并不同步。** 3.6C(80%) – 3.6C 与 4.8C(80%) – 4.8C 两组均为 $Q_1 = 80\%$ 、两阶段倍率相同, 相当于把高倍率维持到较高 SOC。4.8C 组均值 (0.9867) 略低于 3.6C 组 (0.9891), 且下尾更长 (最低约 0.950); 但其中位数反而更高 (0.9960 对 0.9941)。因此“提高恒流倍率则 SOH 下降”只在均值与最差个体意义上可见, 不能由中位数单独下结论, 这两组也不宜当作典型对照。
3. **相近充电时间下仍可拉开寿命。** 长寿命四策略与短寿命策略的平均充电时间均约 10.0 ~ 10.2 min, 说明“时间短”既不是短寿的充分条件, 也不是长寿的必要条件。问题四的优化应同时约束时间与衰减, 而不能只最小化充电时间。
4. **提高总体电流时, 降低 Q_1 有助于保护寿命。** 在 $C_1 > C_2$ 的四组中, 四策略通过调整 Q_1 与 C_1, C_2 使平均充电时间几乎同为 10.04 min, 但 SOH 仍可在窄区间内变化: Q_1 从 67% 降至 19% 时, 高倍率持续时间缩短、后段较低电流覆盖更宽的中高 SOC 区间。四组 SOH 均值均明显高于短寿命策略。更精细的 Q_1 效应与显著性检验留待问题二。
5. **温度、内阻与初始容量不能单独解释长短寿差异。** 短寿命组平均温度最高 (约 35.7°C), 但长寿命的 5.6C 两组温度同样约 35.2 ~ 35.4°C, SOH 却明显更高; 短寿命组平均内阻反而是最低 (约 0.0139 Ω), 初始容量各组均在 1.05 ~ 1.07 Ah。因此在本观测窗口内, 策略形态 (何时施加大电流) 比这三项过程量更能区分典型长、短寿命。
6. **合并组的高离散度提示标签/批次影响。** 统一命名便于后续建模, 但不宜把合并组的中位数直接当作该配方的稳健寿命水平。问题二主分析据此将样本限定为同一实验结构 (dataset_id=3) 的 34 块电池、6 个策略组, 从而自然排除跨批次的 3.6C 组合并组。

上述结论仅基于第 200 次循环的健康保持程度, 样本中若干策略仅 2–4 块满 200 圈电池, 相关关系不能直接等同于因果效应。问题二已在更严格的可比队列上检验策略间差异与 C_1, Q_1, C_2 的统计关联, 并确认两两比较在多重校正后并不显著。

5 问题二: 充电策略参数对循环寿命的影响

5.1 小问一: 明确寿命口径并比较不同策略

问题二的因变量应是 $\text{SOH} = 80\%$ 时的循环寿命 N_{EOL} , 而非第 200 圈 SOH。前 200 圈的 SOH 仅能说明早期健康保持情况, 不能替代终止事件。为与问题三衔接, 本文采用与赛题同源的公开 MATR EOL 标签 [1,4]; 这是一项外部标定数据, 不是附件中直接给出的列。

为避免批次和结构差异造成混杂，主分析只保留 `dataset_id=3`、非问题三测试集、EOL 可用且按原电池去重的记录，共 30 块电池、6 个既有策略。读取标签时先删除全部测试记录，故 9 块问题三测试电池的真实 EOL 未进入问题二的任何检验、图件或回归。策略参数并不是完全析因设计，因此下文的“影响”严格指当前六个配方内的统计关联，不是改变单一参数的因果效应。

表 5 给出寿命概况。‘4.8C(80%)-4.8C’ 的平均 EOL 最高，但只有 3 块且离散较大；‘3.7C(31%)-5.9C’ 最低，只有 2 块。Kruskal-Wallis 总体检验得到 $H = 11.614$ 、 $p = 0.0405$ 、 $\epsilon^2 = 0.276$ ，说明六个策略的 EOL 分布总体存在差异。可是 15 组两两 Mann-Whitney 比较经过 Holm 校正后均未显著；所以不能把表中的任意两个均值写成“已严格证明”的优劣关系。

表 5 同一实验结构下各策略的公开 EOL 寿命汇总（非测试、去重电池）

策略	样本数	平均 EOL / 圈	EOL 中位数 / 圈
4_8C_80PER_4_8C	3	1569.7	1799
5C_67PER_4C	5	1134.8	826
5_3C_54PER_4C	7	1074.0	1036
5_6C_19PER_4_6C	6	947.0	920
5_6C_36PER_4_3C	7	939.0	921
3_7C_31PER_5_9C	2	654.5	654.5

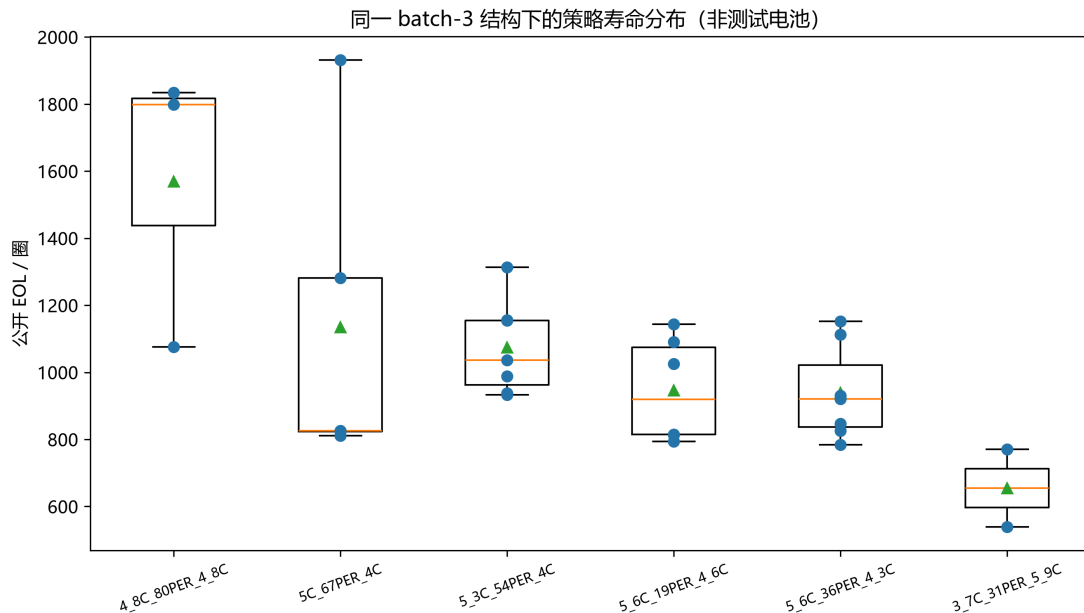


图 6 各既有策略的公开 EOL 分布；点为单体，箱线图不代表可对未观测策略外推

5.2 小问二：参数如何与寿命相联系？

若直接回归 C_1, Q_1, C_2 ，会把策略配方中的共同变化误写成单个参数效应。故在固定 SOC 分界 $Q^* = 50\%$ 下，将两阶段倍率转化为低、高 SOC 窗的电流暴露：

$$E^L = C_1 \min(Q_1, Q^*) + C_2 \max(Q^* - Q_1, 0), \quad (4)$$

$$E^H = C_1 \max(Q_1 - Q^*, 0) + C_2 [80 - \max(Q_1, Q^*)]. \quad (5)$$

例如，‘3.7C(31%)-5.9C’ 在 50–80% SOC 全程采用 5.9C，因而 $E^H = 177$ ，为六组最大；而 ‘5.3C(54%)-4C’ 的 $E^H = 125.2$ 。这样定义不会错误地把 $Q_1 = 80\%$ 的恒流策略判定为“没有高 SOC 电流”。

以每个策略均值为一个设计点、以组内电池数加权，拟合

$$\log N_{\text{EOL}} = \beta_0 + \beta_L E^L + \beta_H E^H + \varepsilon. \quad (6)$$

得到 $\beta_L = -0.0346$ 、 $\beta_H = -0.0340$ ，调整 $R^2 = 0.406$ 。两个系数方向均与“大电流暴露伴随更短寿命”一致，但其描述性 p 值分别为 0.136 和 0.105，不能单独声称显著，也不能比较出高 SOC 的效应一定强于低 SOC。图 7 尤其说明：‘4.8C(80%)-4.8C’ 的高 SOC 暴露不低却有较长寿命，表明高 SOC 倍率只是风险线索，而非唯一决定因素。

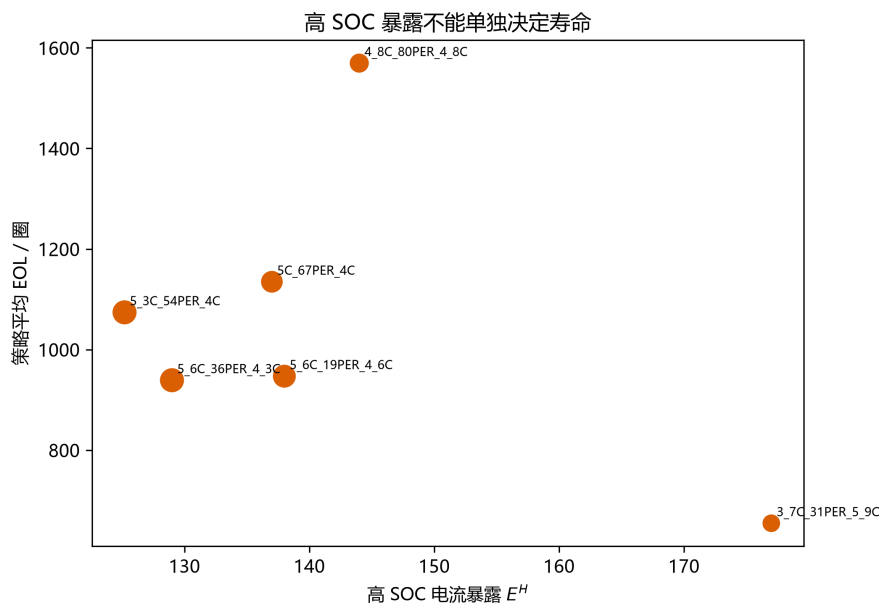


图 7 高 SOC 暴露与策略平均 EOL：存在风险线索，但没有足以支持单因素定律的单调关系

作为反例，将原始 C_1, Q_1, C_2 直接放进同一回归可得到调整 $R^2 = 0.799$ ，看似更好；但只有 6 个设计点和 4 个自由参数，且各变量共同变化，拟合主要是插值。高 R^2 在此并不能证明“提高 C_1 ”或“提高 Q_1 ”会独立延长寿命，故不解释原始参数的单个系数。

5.3 小问三：结论是否能推广到新策略？

为检验式 (6) 是否只是对六个点的事后拟合，依次留出一个完整策略，用其余五个策略重新拟合并预测被留出策略的平均 EOL。留出 ‘5C(67%)-4C’ 和 ‘5.6C(36%)-4.3C’ 时相对误差分别约为 2.2% 与 4.3%，但留出 ‘4.8C(80%)-4.8C’ 时为 36.8%，留出短寿 ‘3.7C(31%)-5.9C’ 时高达 95.8%。这表明两窗口式能描述某些相邻配方，却不能稳定识别远离其余策略的寿命；绝不可把式 (6) 直接用于搜索任意新组合。

因此，问题二支持的稳妥结论是：在本批次样本中，低切换 SOC 后在高 SOC 段继续使用 5.9C 的策略同时表现出最高高 SOC 暴露和最短 EOL；高 SOC 大电流应作为策略设计的风险约束。与此同时，策略总体寿命不同不等于单个参数因果显著，且 ‘4.8C’ 长寿组清楚地提示仍有未测量的电池个体和实验因素。若要识别独立参数效应，需要同批次、正交且有重复的设计实验 [2, 7]。

5.4 与问题三的关系及使用边界

问题三先在附件内验证 “150 圈预测后 50 圈 SOH”，再以非测试公开 EOL 进行隔离标定；本节只使用同一批非测试 EOL 作策略解释。二者共享 SOH = 80% 的终点定义，却绝不使用测试电池真实寿命。这一边界使短期预测精度、EOL 标定误差和策略关联各自有明确的证据来源，不会把公开真值泄漏为模型性能。

6 问题三：基于早期循环的 SOH 与寿命预测

6.1 小问一：预测第 151–200 次循环的 SOH

这部分是一个可以由题目数据直接检验的短期预测任务。9 块测试电池只给出前 150 次循环；其余 40 块电池有完整的 200 次记录。为使验证情形与测试情形一致，对每一块完整电池均人为截断在第 k 圈，并预测紧随其后的 50 圈；训练集和验证集按整块电池划分，避免同一块电池的不同循环同时出现在训练、验证两侧而造成信息泄漏。

设 s_n 为平滑 SOH。首先用第 101–150 圈的最小二乘直线

$$\hat{s}_n = \hat{\alpha} + \hat{\beta}n, \quad n = 151, \dots, 200, \quad (7)$$

外推未来 50 圈，再施加 $\hat{s}_n \leq \hat{s}_{150}$ 的单调不增约束。这一模型没有把不稳定的早期平台直接延伸到很远，只回答 “最近一段的下降趋势在接下来 50 圈如何延续”。

同时，以 C_1, Q_1, C_2 、若干 SOH 检查点、早晚期 SOH 斜率、内阻和温度变化构成特征，训练两隐层小型 MLP，输出第 151–200 圈相对第 150 圈的 SOH 变化量。这样做是为了检验神经网络能否从不同电池间学习共同规律；并不预设深度模型一定优于简单趋势模型 [8]。

表 6 给出 5 折按电池交叉验证结果。对测试电池实际拥有的 150 圈历史，线性尾段模型的 RMSE 为 3.08×10^{-4} ，MAE 为 2.61×10^{-4} ，优于 MLP 的 4.64×10^{-4} 和

3.78×10^{-4} 。因此最终采用式 (7)，而将 MLP 作为短历史窗口下的对照。对只观察到 50 圈的情形，加入策略信息的 MLP 最好；但当观测到 150 圈时，电池自身的近期曲线已包含大部分策略后果，较复杂模型反而更易过拟合 40 块训练电池的个体波动。

表 6 按完整电池 5 折验证：不同可见窗口预测后 50 圈 SOH 的最佳模型

可见至第几圈	最佳模型	RMSE	MAE
50	含策略信息的 MLP	0.000272	0.000227
100	不含策略信息的 MLP	0.000279	0.000234
150	最近 50 圈线性外推	0.000308	0.000261

9 块测试电池的逐圈预测见支撑材料，表 7 压缩展示第 151 圈和第 200 圈结果。预测均保留在 0.96 以上，说明 151–200 圈仍处在早期缓慢衰减阶段；16 号电池的预测下降最明显，应作为后续寿命风险较高的对象。

表 7 测试电池的 SOH 短期预测（最终采用线性尾段模型）

电池编号	$\hat{s}_{151}$	$\hat{s}_{200}$	电池编号	$\hat{s}_{151}$	$\hat{s}_{200}$
2	0.99528	0.99414	5	0.99625	0.99430
9	0.98908	0.98302	10	0.99712	0.99592
11	0.99675	0.99553	14	0.99745	0.99619
16	0.97202	0.96243	24	0.99746	0.99601
25	0.99731	0.99627			

6.2 小问二：估计 $\text{SOH} = 80\%$ 时的循环寿命

短期 SOH 预测准确，并不自动意味着可以可信地预测数百至上千圈后的 EOL。直接以 1–150 圈的 SOH 拟合线性、二次或幂律曲线，再令曲线与 0.8 相交，虽然对第 151–200 圈的 SOH 误差可低至约 0.002，却会给部分电池产生数万乃至更大的寿命外推值。这是因为 LFP 电池的早期平台很平，而平台之后的加速衰减转折在附件中尚未观测到；短期局部拟合无法确定该转折何时发生。故不能把 SOH_{200} 附近的直线直接延长到 80%。

为获得有依据的寿命估计，另设**外部标定**：利用与赛题同源的 MATR 公开寿命标签 [1,4]，只保留 34 块去重后的非测试原电池。以截至第 150 圈可见的 SOH 检查点、四段斜率、内阻/温度变化及策略参数为输入，在 $\log N_{\text{EOL}}$ 空间拟合带 L_2 正则的岭回归：

$$\log \hat{N}_{\text{EOL}} = \beta_0 + \sum_j \beta_j x_j. \quad (8)$$

对数变换使 500 圈和 2000 圈电池的相对误差获得较均衡的权重。每次留一验证均不使用被留出电池的 EOL；岭回归的留一验证 MAE 为 236.77 圈，中位绝对百分误差为 11.75%。随机森林和极端随机树的中位相对误差分别为 18.05% 和 15.64%，故不采用。

特别地，外部标签只用于本小问的标定与审计，9 块测试电池的公开 EOL 标签从未进入训练、调参或预测输入；其标签也不用于本文报分。2 号和 5 号对应同一原始电池的接续记录，故先作记录级预测，再在对数寿命上取平均，强制给出同一 EOL。表 8 是据此得到的估计；它应理解为“结合公开同源先验的寿命风险估计”，其不确定性约为数百圈，不能写成精确的真实寿命。

表 8 测试电池的 EOL 估计（外部标定，不参与题内短期 SOH 评分）

电池编号	2	5	9	10	11	14	16	24	25
$\hat{N}_{\text{EOL}} / \text{圈}$	1056	1056	639	1099	1169	918	585	964	1246

6.3 小问三：早期窗口和策略信息的作用

表 6 给出三个清晰结论：其一，只有 50 圈历史时，策略参数能补充尚未显现的退化趋势；其二，增加至 100 圈后，SOH 轨迹本身已携带主要信息；其三，达到 150 圈时，局部趋势模型更稳健。这说明“是否应加入策略特征”不是固定答案，而取决于可见历史的长度。对实际部署而言，前 50 圈可优先采用群体模型和策略先验；运行至 150 圈后，应优先相信该电池最近的观测趋势，并以较保守的区间报告 EOL 风险 [7,9]。

6.4 可复现性与适用边界

短期 SOH 验证和 EOL 外部标定是两套不能混淆的证据。前者完全由赛题 49 块电池的已知前 200 圈实现，代码为 `src/A/task3_window_ablation.py`；后者依赖公开 MATR 标签，代码为 `src/A/task3_external_eol_calibration.py`，并明确隔离测试标签。逐圈预测、交叉验证明细和 EOL 留一预测均放在 `A/问题三/output/`。若竞赛规则禁止使用同源公开 EOL 标签，则应保留前者作为可评分答案，并把表 8 降为外部合理性分析，而不是声称附件单独足以确定 EOL。

7 问题四：兼顾充电时间与寿命衰减的策略优化

待完成 / 框架占位

本节为建模框架。优化应优先在问题一统一后的 7 种实验策略内比较；若连续优化，参数须限制在实验数据合理邻域并说明依据 [10–13]。

7.1 决策变量与目标

决策变量 $x = (C_1, Q_1, C_2)$ 。目标为在较短充电时间下降低衰减速率或延长寿命：

$$\min_x F(x) = \begin{bmatrix} T_{\text{charge}}(x) \\ \text{DecayRate}(x) \end{bmatrix} \quad \text{或} \quad \min_x w_1 T_{\text{charge}}(x) + w_2 \text{DecayRate}(x). \quad (9)$$

7.2 充电时间模型（拟采用）

分段恒流近似

$$T_{\text{charge}} \approx \frac{Q_1}{C_1} + \frac{0.8 - Q_1}{C_2} + T_{\text{CV}}, \quad (10)$$

并用 `mean_chargetime` 回归校准。问题一散点图已显示充电时间与策略分组相关，可用于检验该近似。

7.3 寿命衰减模型

沿用问题二回归或问题三预测模型，得到 $\text{DecayRate}(x)$ 或 $\text{SOH}_{200}(x)$ 。在已有策略上先做离散 Pareto 比较，再考虑邻域连续搜索。

7.4 约束与邻域

建议参数盒约束取自样本极值（以最终校核为准）： $C_1 \in [3.6, 5.6]$ ， $Q_1 \in [19, 80]$ ， $C_2 \in [3.6, 5.9]$ 。超出实验凸包的组合须标明外推风险。

7.5 推荐策略与对比（待填）

给出推荐 (C_1^*, Q_1^*, C_2^*) ，并与问题一的典型长寿命策略（如 5C_67PER_4C、5_3C_54PER_4C）及短寿命策略 3_7C_31PER_5_9C 对比充电时间与寿命代理。说明适用范围（LFP、两阶段快充、早期循环窗口）与不可外推情形。

8 结论与展望

已完成结果（仓库现有成果）

问题一已形成的结论：

- 循环数据经 IQR 清洗后可用于策略对比；原始 9 种策略统一为 7 类。
- 在满 200 圈的 40 块电池中，第 200 次循环 SOH 均值为 0.9914，尚未达到 80% EOL。
- 按中位数会把 4.8C 合并组标为最长，但其组内离散大、充电时间分两簇，故不以该组为典型；典型长寿命满足 $C_1 > C_2$ （如 5C(67%) – 4C 等），充电时间约 10.04 min；典型短寿命为 3.7C(31%) – 5.9C ($C_1 < C_2$)。
- 相近充电时间下，高 SOC 区间高倍率会明显加快衰减； $Q_1 = 80\%$ 的恒流合并组离散大，提高倍率时均值与下尾变差但中位数并不同步，故不作典型对照；提高电流时降低 Q_1 有助于保持 SOH。
- 温度、内阻与初始容量不能单独解释长短寿差异。

已完成结果（仓库现有成果）

问题二结论：

- 在同一实验结构的 30 块去重非测试电池中，六种策略的公开 EOL 分布存在总体差异（Kruskal-Wallis: $H = 11.614, p = 0.0405$ ），但 Holm 校正后的两两比较均未显著，故不对具体策略作确定性优劣排序。
- 低切换 SOC 后在高 SOC 段使用 5.9C 的策略同时具有最高高 SOC 暴露和最低平均 EOL；这是一条值得规避的风险线索，但不能从六个非正交配方推出单一倍率的因果效应。
- 两窗口 $\log N_{\text{EOL}}$ 模型的留一策略误差最高为 95.8%，故不用它直接生成新策略；策略解释限于现有配方附近。

已完成结果（仓库现有成果）

问题三结论：

- 对“前 150 圈预测后 50 圈”，最近 50 圈线性尾段外推的按电池 5 折验证 RMSE 为 0.000308、MAE 为 0.000261，优于小型 MLP，故作为最终 SOH 预测模型。
- 早期 SOH 平台不能可靠识别后期加速衰减，不能仅凭附件的 150 圈数据将线性/幂律曲线直接延长到 80% EOL。
- 使用公开同源的非测试 EOL 进行隔离标定后，岭回归留一验证中位绝对百分误差为 11.75%。测试电池中 16 号的估计 EOL 最短（约 585 圈），应优先监测；这些数值是外部先验下的风险估计，而非已知真值。

待完成 / 框架占位

补充问题四推荐策略及与长短寿命策略的对比；讨论模型局限与可推广范围。

参考文献

参考文献

- [1] Kristen A. Severson, Peter M. Attia, Norman Jin, Nicholas Perkins, Benben Jiang, Zi Yang, Michael H. Chen, Muratahan Aykol, Patrick K. Herring, Dimitrios Fraggedakis, Martin Z. Bazant, Stephen J. Harris, William C. Chueh, and Richard D. Braatz. Data-driven prediction of battery cycle life before capacity degradation. *Nature Energy*, 4(5):383–391, 2019. 本题主数据集来源；问题 1-4 必引。
- [2] Yayuan Liu, Yangying Zhu, and Yi Cui. Challenges and opportunities towards fast-charging battery materials. *Nature Energy*, 4:540–550, 2019. 快充机理综述；问题 1 机制解释。
- [3] Shabbir Ahmed, Ira Bloom, Andrew N. Jansen, Tanvir R. Tanim, Eric J. Dufek, Ahmad Pesaran, et al. Enabling fast charging – A battery technology gap assessment. *Journal of Power Sources*, 367:250–262, 2017. 快充技术缺口评估；问题 1/2 背景。
- [4] MIT–Stanford–TRI battery dataset. <https://data.matr.io/1/>. Severson 2019 配套公开数据；本题主数据。
- [5] Matthieu Dubarry and Bor Yann Liaw. Identify capacity fading mechanism in a commercial LiFePO₄ cell. *Journal of Power Sources*, 194:541–549, 2009. LFP 衰减机理；问题 1/2。

- [6] Y. Ma et al. An aging-optimized state-of-charge-controlled multi-stage constant current (MCC) fast charging algorithm for commercial Li-ion battery based on three-electrode measurements. *Batteries*, 10(8):267, 2024. 多段恒流快充；问题 2/4.
- [7] Peter M. Attia, Aditya Grover, Norman Jin, Kristen A. Severson, et al. Statistical learning for accurate and interpretable battery lifetime prediction. *Journal of The Electrochemical Society*, 2021. 问题 3 特征工程与可解释预测.
- [8] Yongzhi Zhang, Rui Xiong, Hongwen He, and Michael G. Pecht. Long short-term memory recurrent neural network for remaining useful life prediction of lithium-ion batteries. *IEEE Transactions on Vehicular Technology*, 67(7):5695–5705, 2018. LSTM-RNN and Monte Carlo probabilistic RUL prediction.
- [9] Yongzhi Zhang, Rui Xiong, Hongwen He, and Michael G. Pecht. Lithium-ion battery remaining useful life prediction with box-cox transformation and monte carlo simulation. *IEEE Transactions on Industrial Electronics*, 66(2):1585–1597, 2019. Box-Cox transformation and Monte Carlo simulation for online RUL prediction.
- [10] Peter M. Attia, Aditya Grover, Norman Jin, Kristen A. Severson, Todor M. Markov, Yang Liao, Michael H. Chen, Bryan Cheong, Nicholas Perkins, Zi Yang, Patrick K. Herring, Muratahan Aykol, Stephen J. Harris, Richard D. Braatz, Stefano Ermon, and William C. Chueh. Closed-loop optimization of fast-charging protocols for batteries with machine learning. *Nature*, 578:397–402, 2020. 问题 3 早期预测 + 问题 4 贝叶斯优化快充协议.
- [11] Gregory L. Plett and M. Scott Trimboli. *Battery Management Systems, Volume III: Physics-Based Methods*. Artech House, 2022. 物理模型与快充优化；问题 4.
- [12] Peter I. Frazier. A tutorial on Bayesian optimization, 2018. 问题 4 贝叶斯优化方法.
- [13] Kalyanmoy Deb. *Multi-Objective Optimization Using Evolutionary Algorithms*. Wiley, 2001. 问题 4 多目标优化 / NSGA-II.

A 支撑材料文件列表

按赛制，支撑材料单独打包为压缩文件（RAR/ZIP，不超过 20 MB），本附录列出其内容。赛题原始数据 `data/battery_summary.csv`、`data/cycle_train.csv` 不作为“自主查阅数据”重复提交，但建模程序依赖它们运行。

问题四的脚本、中间表与预测结果待生成后追加本表。若最终提交时上述文件均已纳入压缩包，则与论文结论对应；源程序运行方式见附录 B。

表 9 支撑材料拟包含的文件

路径	说明
src/A/task1_pipeline.py	问题一完整可运行流水线
src/A/SOH_plot.py	SOH 曲线绘制模块
A/问题一/AGENTS.md	问题一任务说明（提示词）
A/问题一/log_1.md-log_3.md	运行日志
A/问题一/总结.md	问题一结论摘要
A/问题一/output/A/cycle_train_cleaned.csv	清洗后循环数据
A/问题一/output/A/charge_policy.csv	统一后的电池-策略表
A/问题一/output/A/battery_SOH_charge_200.csv	200 圈 SOH 与充电时间
A/问题一/output/A/policy.csv	7 类策略汇总
A/问题一/output/A/doc/1.md	策略统计报告
A/问题一/output/A/acceptance_3.md	第三次迭代验收
A/问题一/output/A/*.png	问题一图件
A/问题一/SOH/	49 张单体曲线与总图 SOH_ALL
src/A/task2_statistical_analysis.py	问题二统计分析脚本
src/A/task2_eol_analysis.py	问题二：非测试公开 EOL 的策略寿命分析
output/A/question2/	问题二检验表、回归诊断与图件
A/问题二/output/eol_lifetime/	EOL 策略汇总、检验、留一策略与图件（测试标签隔离）
A/问题二/AGENTS.md、log_1.md	问题二任务说明与日志
src/A/task3_window_ablation.py	问题三：SOH 窗口验证与 151-200 圈预测
src/A/task3_external_eol_calibration.py	问题三：非测试公开 EOL 的隔离标定脚本
A/问题三/output/window_ablation/	交叉验证表、逐圈 SOH 预测与图件
A/问题三/output/external_eol_calibration/	窗口验证、特征表与 EOL 估计（外部先验，测试标签隔离）
references/	参考文献导读与 BibTeX
AI 使用记录/	AI 使用详情底稿
paper/	本 LaTeX 工程

B 建模源程序

本节给出问题一至问题三可运行源代码。运行环境：Python 3, 依赖 numpy、pandas、matplotlib; 问题二另需 scipy、statsmodels; 问题三的神经网络对照需要 PyTorch, 外部 EOL 标定另需 scikit-learn。在项目根目录执行：

```
1 python src/A/task1_pipeline.py
2 python src/A/task2_statistical_analysis.py
3 python src/A/task2_eol_analysis.py
4 python src/A/task3_window_ablation.py
5 python src/A/task3_external_eol_calibration.py
```

可选参数 `--iqr-factor` (默认 1.5)。单独绘制指定电池 SOH 曲线：

```
1 python src/A/SOH_plot.py --battery-id 1
```

B.1 SOH_plot.py

Listing 1 SOH 曲线绘制脚本

```
1  """绘制指定电池的 SOH 曲线。"""
2
3  from __future__ import annotations
4
5  import argparse
6  from pathlib import Path
7
8  import matplotlib
9
10 matplotlib.use("Agg")
11
12 import matplotlib.font_manager as font_manager
13 import matplotlib.pyplot as plt
14 import numpy as np
15 import pandas as pd
16
17
18 PROJECT_ROOT = Path(__file__).resolve().parents[2]
19
20
21 def configure_chinese_font() -> str:
22     """选择系统中可用的中文字体，并返回字体名称。"""
23     installed = {font.name for font in font_manager.fontManager.ttflist}
24     candidates = [
25         "Microsoft YaHei",
```

```

26     "SimHei",
27     "Noto Sans CJK SC",
28     "Source Han Sans SC",
29     "Arial Unicode MS",
30 ]
31 selected = next((name for name in candidates if name in installed), "DejaVu
    Sans")
32 plt.rcParams["font.sans-serif"] = [selected, "DejaVu Sans"]
33 plt.rcParams["axes.unicode_minus"] = False
34 return selected
35
36
37 def plot_battery_soh(
38     cycles: pd.DataFrame,
39     battery_id: int,
40     output_path: Path,
41 ) -> None:
42     """绘制指定电池的平滑 SOH 曲线。"""
43     required = {"battery_id", "cycle", "SOH_smooth"}
44     missing = required.difference(cycles.columns)
45     if missing:
46         raise ValueError(f"循环数据缺少字段: {sorted(missing)}")
47
48     battery = cycles.loc[cycles["battery_id"] == battery_id].sort_values("cycle"
49 )
50     if battery.empty:
51         raise ValueError(f"未找到 battery_id={battery_id} 的数据")
52
53     configure_chinese_font()
54     fig, ax = plt.subplots(figsize=(10, 6))
55     ax.plot(
56         battery["cycle"],
57         battery["SOH_smooth"],
58         color="#4C78A8",
59         linewidth=2.2,
60         label="平滑 SOH (SOH_smooth) ",
61     )
62     ax.set_title(f"{battery_id} 号电池 SOH 曲线")
63     ax.set_xlabel("循环次数")
64     ax.set_ylabel("电池健康状态 (SOH) ")
65     ax.grid(True, color="#D9D9D9", linewidth=0.7, alpha=0.7)
66     ax.legend(loc="upper right")
67     fig.tight_layout()
68     output_path.parent.mkdir(parents=True, exist_ok=True)

```

```

68     fig.savefig(output_path, dpi=300, bbox_inches="tight")
69     plt.close(fig)
70
71
72 def plot_all_battery_soh(cycles: pd.DataFrame, output_path: Path) -> None:
73     """用蓝到红渐变绘制全部电池的平滑 SOH 曲线。"""
74     required = {"battery_id", "cycle", "SOH_smooth"}
75     missing = required.difference(cycles.columns)
76     if missing:
77         raise ValueError(f"循环数据缺少字段: {sorted(missing)}")
78
79     battery_ids = sorted(int(value) for value in cycles["battery_id"].unique())
80     if not battery_ids:
81         raise ValueError("循环数据中没有电池记录")
82
83     configure_chinese_font()
84     fig, ax = plt.subplots(figsize=(14, 8))
85     colormap = matplotlib.colormaps.get_cmap("coolwarm")
86     colors = colormap(np.linspace(0, 1, len(battery_ids)))
87     for color, battery_id in zip(colors, battery_ids):
88         battery = cycles.loc[cycles["battery_id"] == battery_id].sort_values("
            cycle")
89         ax.plot(
90             battery["cycle"],
91             battery["SOH_smooth"],
92             color=color,
93             linewidth=1.05,
94             alpha=0.78,
95         )
96
97     ax.set_title("全部电池平滑 SOH 曲线")
98     ax.set_xlabel("循环次数")
99     ax.set_ylabel("电池健康状态 (SOH_smooth)")
100    ax.grid(True, color="#D9D9D9", linewidth=0.6, alpha=0.6)
101    ax.legend(
102        handles=[
103            plt.Line2D([0], [0], color=colors[0], linewidth=2, label=f"低编号: 电
                池 {battery_ids[0]}"),
104            plt.Line2D([0], [0], color=colors[-1], linewidth=2, label=f"高编号: 电
                池 {battery_ids[-1]}"),
105        ],
106        loc="upper right",
107        title="颜色由蓝到红",
108    )

```

```

109     scalar_mappable = plt.cm.ScalarMappable(
110         cmap=colormap,
111         norm=plt.Normalize(vmin=battery_ids[0], vmax=battery_ids[-1]),
112     )
113     scalar_mappable.set_array([])
114     colorbar = fig.colorbar(scalar_mappable, ax=ax, pad=0.015)
115     colorbar.set_label("battery_id")
116     fig.tight_layout()
117     output_path.parent.mkdir(parents=True, exist_ok=True)
118     fig.savefig(output_path, dpi=300, bbox_inches="tight")
119     plt.close(fig)
120
121
122 def parse_args() -> argparse.Namespace:
123     parser = argparse.ArgumentParser(description="绘制指定电池的 SOH 曲线")
124     parser.add_argument("--battery-id", type=int, default=1, help="电池编号，默认
125         为 1")
126     parser.add_argument(
127         "--input",
128         type=Path,
129         default=PROJECT_ROOT / "output" / "A" / "cycle_train_cleaned.csv",
130         help="循环数据 CSV 路径",
131     )
132     parser.add_argument(
133         "--output",
134         type=Path,
135         default=PROJECT_ROOT / "output" / "A" / "SOH_1_example.png",
136         help="输出图像路径",
137     )
138     return parser.parse_args()
139
140 def main() -> None:
141     args = parse_args()
142     cycles = pd.read_csv(args.input)
143     plot_battery_soh(cycles, args.battery_id, args.output)
144     print(f"已生成: {args.output}")
145
146
147 if __name__ == "__main__":
148     main()

```

B.2 task1_pipeline.py

Listing 2 问题一分析流水线

```
1  """问题一：数据清洗、策略提取、200 次循环汇总与绘图。"""
2
3  from __future__ import annotations
4
5  import argparse
6  from collections import Counter
7  from datetime import datetime
8  from pathlib import Path
9
10 import matplotlib
11
12 matplotlib.use("Agg")
13
14 import matplotlib.pyplot as plt
15 from matplotlib.patches import Patch
16 import numpy as np
17 import pandas as pd
18
19 from SOH_plot import configure_chinese_font, plot_all_battery_soh,
    plot_battery_soh
20
21
22 PROJECT_ROOT = Path(__file__).resolve().parents[2]
23 MEASUREMENT_COLUMNS = ["capacity", "SOH", "SOH_smooth", "chargetime", "IR", "
    Tavg"]
24
25
26 def normalize_policy_name(policy: str) -> str:
27     """按第三次迭代规则合并策略并删除 NEWSTRUCTURE 后缀。"""
28     normalized = str(policy).strip()
29     direct_mapping = {
30         "80PER_3_6C": "3_6C-80PER_3_6C",
31         "3_6C-80PER_3_6C": "3_6C-80PER_3_6C",
32         "4_8C_80PER_4_8C_NEWSTRUCTURE": "4_8C_80PER_4_8C",
33         "4_8C_80PER_4_8C": "4_8C_80PER_4_8C",
34     }
35     if normalized in direct_mapping:
36         return direct_mapping[normalized]
37     upper = normalized.upper()
38     if upper.endswith("_NEWSTRUCTURE"):
39         normalized = normalized[:-len("_NEWSTRUCTURE")]
```



```

40     return normalized
41
42
43 def normalize_policies(
44     summary: pd.DataFrame,
45     cycles: pd.DataFrame,
46 ) -> tuple[pd.DataFrame, pd.DataFrame, dict[str, str]]:
47     """在处理副本中统一两个数据表的策略名称，原始数据保持不变。"""
48     normalized_summary = summary.copy()
49     normalized_cycles = cycles.copy()
50     original_names = sorted(str(value) for value in summary["policy"].unique())
51     mapping = {name: normalize_policy_name(name) for name in original_names}
52     normalized_summary["policy"] = normalized_summary["policy"].map(mapping)
53     normalized_cycles["policy"] = normalized_cycles["policy"].map(mapping)
54     if normalized_summary["policy"].isna().any() or normalized_cycles["policy"].
55         isna().any():
56         raise ValueError("策略统一命名过程中产生空值")
57     return normalized_summary, normalized_cycles, mapping
58
59 def validate_inputs(summary: pd.DataFrame, cycles: pd.DataFrame) -> None:
60     summary_required = {"battery_id", "policy", "initial_capacity", "
61         prediction_test"}
62     cycle_required = {
63         "battery_id",
64         "cycle",
65         "capacity",
66         "SOH",
67         "SOH_smooth",
68         "chargetime",
69         "IR",
70         "Tavg",
71         "policy",
72     }
73     summary_missing = summary_required.difference(summary.columns)
74     cycle_missing = cycle_required.difference(cycles.columns)
75     if summary_missing or cycle_missing:
76         raise ValueError(
77             f"输入字段不完整; battery_summary 缺少 {sorted(summary_missing)}, "
78             f"cycle_train 缺少 {sorted(cycle_missing)}"
79         )
80     if summary["battery_id"].duplicated().any():
81         raise ValueError("battery_summary.csv 中 battery_id 不唯一")
82     if cycles.duplicated(["battery_id", "cycle"]).any():

```

```

82     raise ValueError("cycle_train.csv 中存在重复的 battery_id + cycle")
83 if set(summary["battery_id"]) != set(cycles["battery_id"]):
84     raise ValueError("两个输入文件的 battery_id 集合不一致")
85
86 policy_check = cycles.merge(
87     summary[["battery_id", "policy"]],
88     on="battery_id",
89     suffixes=("_cycle", "_summary"),
90     validate="many_to_one",
91 )
92 if (policy_check["policy_cycle"] != policy_check["policy_summary"]).any():
93     raise ValueError("cycle_train.csv 与 battery_summary.csv 的策略字段不一致")
94
95
96 def iqr_clean_by_battery(
97     cycles: pd.DataFrame,
98     columns: list[str],
99     factor: float = 1.5,
100 ) -> tuple[pd.DataFrame, list[dict[str, float | int | str]]]:
101     """逐电池、逐字段进行 IQR 检测，并用最近前后正常值的均值替换。"""
102     cleaned = cycles.sort_values(["battery_id", "cycle"]).copy()
103     audit: list[dict[str, float | int | str]] = []
104
105     for battery_id, group in cycles.sort_values(["battery_id", "cycle"]).groupby(
106         "battery_id"):
107         for column in columns:
108             values = group[column].to_numpy(dtype=float)
109             finite_values = values[np.isfinite(values)]
110             if finite_values.size == 0:
111                 continue
112
113             q1, q3 = np.quantile(finite_values, [0.25, 0.75])
114             iqr = q3 - q1
115             if np.isclose(iqr, 0.0):
116                 continue
117
118             lower = q1 - factor * iqr
119             upper = q3 + factor * iqr
120             outlier_mask = (~np.isfinite(values)) | (values < lower) | (values >
121                 upper)
122             normal_positions = np.flatnonzero(~outlier_mask)
123
124             for position in np.flatnonzero(outlier_mask):
125                 left_candidates = normal_positions[normal_positions < position]

```

```

123         right_candidates = normal_positions[normal_positions > position]
124         neighbors: list[float] = []
125         if left_candidates.size:
126             neighbors.append(float(values[left_candidates[-1]]))
127         if right_candidates.size:
128             neighbors.append(float(values[right_candidates[0]]))
129         if not neighbors:
130             continue
131
132         replacement = float(np.mean(neighbors))
133         row_index = group.index[position]
134         cleaned.at[row_index, column] = replacement
135         audit.append(
136             {
137                 "battery_id": int(battery_id),
138                 "cycle": int(group.iloc[position]["cycle"]),
139                 "column": column,
140                 "original": float(values[position]),
141                 "replacement": replacement,
142                 "lower_bound": float(lower),
143                 "upper_bound": float(upper),
144             }
145         )
146
147     return cleaned.sort_values(["battery_id", "cycle"]), audit
148
149
150 def build_policy_table(summary: pd.DataFrame) -> pd.DataFrame:
151     return (
152         summary[["battery_id", "policy"]]
153         .sort_values("battery_id")
154         .reset_index(drop=True)
155     )
156
157
158 def build_cycle_200_summary(
159     cleaned_cycles: pd.DataFrame,
160     policy_table: pd.DataFrame,
161 ) -> pd.DataFrame:
162     max_cycle = cleaned_cycles.groupby("battery_id")["cycle"].max()
163     eligible_ids = max_cycle[max_cycle == 200].index
164     eligible = cleaned_cycles[cleaned_cycles["battery_id"].isin(eligible_ids)]
165
166     cycle_200 = eligible.loc[eligible["cycle"] == 200, ["battery_id", "SOH"]]

```

```

167     if cycle_200["battery_id"].duplicated().any() or len(cycle_200) != len(
168         eligible_ids):
169         raise ValueError("达到 200 次循环的电池没有且仅有一条 cycle=200 记录")
170
171     charge_mean = (
172         eligible.groupby("battery_id", as_index=False)["chargetime"]
173         .mean()
174         .rename(columns={"chargetime": "charge_time_mean"})
175     )
176     result = (
177         cycle_200.merge(policy_table, on="battery_id", validate="one_to_one")
178         .merge(charge_mean, on="battery_id", validate="one_to_one")
179         [["battery_id", "policy", "SOH", "charge_time_mean"]]
180         .sort_values("battery_id")
181         .reset_index(drop=True)
182     )
183     return result
184
185 def build_policy_summary(
186     data: pd.DataFrame,
187     policy_order: list[str],
188     critical_soh_low: float,
189     critical_soh_high: float,
190     summary: pd.DataFrame,
191     cleaned_cycles: pd.DataFrame,
192 ) -> pd.DataFrame:
193     """汇总统一策略的最终 SOH、充电时间和其他实验参数。"""
194     classified = data.assign(
195         lifetime_group=np.select(
196             [data["SOH"] >= critical_soh_high, data["SOH"] <= critical_soh_low],
197             ["长寿命", "短寿命"],
198             default="中等寿命",
199         )
200     )
201     grouped = classified.groupby("policy", sort=False)
202     result = grouped.agg(
203         battery_count=("battery_id", "size"),
204         SOH_min=("SOH", "min"),
205         SOH_max=("SOH", "max"),
206         SOH_mean=("SOH", "mean"),
207         SOH_median=("SOH", "median"),
208         charge_time_mean=("charge_time_mean", "mean"),
209         charge_time_min=("charge_time_mean", "min"),

```

```

210     charge_time_max=("charge_time_mean", "max"),
211 ).reindex(policy_order)
212 result["battery_ids"] = grouped["battery_id"].apply(
213     lambda values: ",".join(str(int(value)) for value in sorted(values))
214 ).reindex(policy_order)
215
216 eligible_ids = set(int(value) for value in data["battery_id"])
217 battery_cycle_metrics = (
218     cleaned_cycles.loc[cleaned_cycles["battery_id"].isin(eligible_ids)]
219     .groupby(["battery_id", "policy"], as_index=False)
220     .agg(IR_mean=("IR", "mean"), Tavg_mean=("Tavg", "mean"))
221 )
222 cycle_metrics = battery_cycle_metrics.groupby("policy").agg(
223     IR_mean=("IR_mean", "mean"),
224     Tavg_mean=("Tavg_mean", "mean"),
225 ).reindex(policy_order)
226 result = result.join(cycle_metrics)
227
228 eligible_summary = summary.loc[summary["battery_id"].isin(eligible_ids)]
229 capacity_metrics = eligible_summary.groupby("policy").agg(
230     initial_capacity_mean=("initial_capacity", "mean"),
231     initial_capacity_min=("initial_capacity", "min"),
232     initial_capacity_max=("initial_capacity", "max"),
233 ).reindex(policy_order)
234 result = result.join(capacity_metrics)
235
236 def parameter_values(series: pd.Series) -> str:
237     values = [f"{float(value):g}" for value in sorted(series.dropna().unique
238         ())]
239     if series.isna().any():
240         values.append("缺失")
241     return "/".join(values)
242
243 for column in ["C1", "Q1", "C2"]:
244     if column in eligible_summary.columns:
245         result[f"{column}_values"] = (
246             eligible_summary.groupby("policy")[column].apply(parameter_values)
247             .reindex(policy_order)
248         )
249
250 group_counts = (
251     classified.groupby(["policy", "lifetime_group"])
252     .size()
253     .unstack(fill_value=0)
254     .reindex(policy_order, fill_value=0)

```

```

252 )
253 for group_name, column_name in [
254     ("长寿命", "long_battery_count"),
255     ("中等寿命", "middle_battery_count"),
256     ("短寿命", "short_battery_count"),
257 ]:
258     result[column_name] = group_counts.get(group_name, pd.Series(0, index=
259         result.index)).astype(int)
260
261 result["lifetime_label"] = np.select(
262     [result["SOH_median"] >= critical_soh_high, result["SOH_median"] <=
263         critical_soh_low],
264     ["长寿命策略", "短寿命策略"],
265     default="中等寿命策略",
266 )
267 max_median = result["SOH_median"].max()
268 min_median = result["SOH_median"].min()
269 result["longest_strategy"] = np.where(np.isclose(result["SOH_median"],
270     max_median), "是", "否")
271 result["shortest_strategy"] = np.where(np.isclose(result["SOH_median"],
272     min_median), "是", "否")
273 result["SOH_range"] = result.apply(lambda row: f"{row['SOH_min']:.6f} - {row
274     ['SOH_max']:.6f}", axis=1)
275 result["charge_time_range"] = result.apply(
276     lambda row: f"{row['charge_time_min']:.6f} - {row['charge_time_max']:.6f}"
277     , axis=1
278 )
279 return result.reset_index()
280
281 def save_soh_histogram(
282     data: pd.DataFrame,
283     output_path: Path,
284     critical_soh_low: float,
285     critical_soh_high: float,
286 ) -> None:
287     values = data["SOH"].to_numpy(dtype=float)
288     bin_edges = np.histogram_bin_edges(values, bins="fd")
289     if len(bin_edges) < 7:
290         bin_edges = np.linspace(values.min(), values.max(), 8)
291
292     fig, ax = plt.subplots(figsize=(10, 6))
293     ax.hist(
294         values,

```

```

290     bins=bin_edges,
291     color="#4C78A8",
292     edgecolor="white",
293     linewidth=1.0,
294     label="电池数量",
295 )
296 ax.axvline(
297     critical_soh_high,
298     color="#D62728",
299     linestyle="--",
300     linewidth=2,
301     label=f"上四分位线 (Q3) = {critical_soh_high:.4f}",
302 )
303 ax.axvline(
304     critical_soh_low,
305     color="#1F77B4",
306     linestyle="--",
307     linewidth=2,
308     label=f"下四分位线 (Q1) = {critical_soh_low:.4f}",
309 )
310 margin = max((values.max() - values.min()) * 0.08, 0.003)
311 ax.set_xlim(values.min() - margin, values.max() + margin)
312 ax.set_title("第 200 次循环 SOH 分布")
313 ax.set_xlabel("第 200 次循环的 SOH")
314 ax.set_ylabel("电池数量")
315 ax.grid(axis="y", color="#D9D9D9", linewidth=0.7, alpha=0.7)
316 ax.legend(loc="upper right")
317 fig.tight_layout()
318 fig.savefig(output_path, dpi=300, bbox_inches="tight")
319 plt.close(fig)
320
321
322 def save_soh_charge_scatter(
323     data: pd.DataFrame,
324     output_path: Path,
325     policy_order: list[str],
326 ) -> None:
327     fig, ax = plt.subplots(figsize=(13, 8))
328     color_map = matplotlib.colormaps.get_cmap("tab10")
329     colors = color_map(np.linspace(0, 1, max(len(policy_order), 2)))
330     for color, policy in zip(colors, policy_order):
331         group = data.loc[data["policy"] == policy]
332         if group.empty:
333             continue

```

```

334     ax.scatter(
335         group["SOH"],
336         group["charge_time_mean"],
337         s=64,
338         color=color,
339         edgecolors="white",
340         linewidths=0.8,
341         alpha=0.88,
342         label=policy,
343     )
344     ax.set_title("第 200 次循环 SOH 与平均充电时间")
345     ax.set_xlabel("第 200 次循环的 SOH")
346     ax.set_ylabel("平均充电时间 (min) ")
347     ax.grid(True, color="#D9D9D9", linewidth=0.7, alpha=0.7)
348     ax.legend(loc="upper right", fontsize=8, framealpha=0.92, title="充电策略",
349             title_fontsize=9)
350     fig.tight_layout()
351     fig.savefig(output_path, dpi=300, bbox_inches="tight")
352     plt.close(fig)
353
354 def save_policy_boxplot(
355     data: pd.DataFrame,
356     output_path: Path,
357     critical_soh_low: float,
358     critical_soh_high: float,
359     policy_order: list[str],
360 ) -> None:
361     available_order = [policy for policy in policy_order if policy in set(data["
362         policy"])]
363     groups = [data.loc[data["policy"] == policy, "SOH"].to_numpy() for policy in
364         available_order]
365
366     fig, ax = plt.subplots(figsize=(16, 8))
367     box = ax.boxplot(
368         groups,
369         tick_labels=available_order,
370         patch_artist=True,
371         widths=0.62,
372         medianprops={"color": "#1F1F1F", "linewidth": 1.5},
373         whiskerprops={"color": "#555555"},
374         capprops={"color": "#555555"},
375         flierprops={
376             "marker": "o",

```



```

375         "markerfacecolor": "white",
376         "markeredgecolor": "#4C78A8",
377         "markersize": 5,
378     },
379 )
380 for patch in box["boxes"]:
381     patch.set_facecolor("#9ECAE1")
382     patch.set_edgecolor("#4C78A8")
383
384 ax.axhline(
385     critical_soh_high,
386     color="#D62728",
387     linestyle="--",
388     linewidth=2,
389 )
390 ax.axhline(
391     critical_soh_low,
392     color="#1F77B4",
393     linestyle="--",
394     linewidth=2,
395 )
396 all_values = data["SOH"].to_numpy(dtype=float)
397 margin = max((all_values.max() - all_values.min()) * 0.08, 0.003)
398 ax.set_ylim(all_values.min() - margin, all_values.max() + margin)
399 ax.set_title("不同充电策略下的第 200 次循环 SOH 分布")
400 ax.set_xlabel("充电策略")
401 ax.set_ylabel("使用寿命指标: 第 200 次循环的 SOH")
402 ax.tick_params(axis="x", labelrotation=40)
403 for label in ax.get_xticklabels():
404     label.set_horizontalalignment("right")
405 ax.grid(axis="y", color="#D9D9D9", linewidth=0.7, alpha=0.7)
406 legend_handles = [
407     Patch(facecolor="#9ECAE1", edgecolor="#4C78A8", label="策略内 SOH 分布"),
408     plt.Line2D(
409         [0], [0], color="#D62728", linestyle="--", linewidth=2,
410         label=f"上四分位线 (Q3)={critical_soh_high:.4f}",
411     ),
412     plt.Line2D(
413         [0], [0], color="#1F77B4", linestyle="--", linewidth=2,
414         label=f"下四分位线 (Q1)={critical_soh_low:.4f}",
415     ),
416 ]
417 ax.legend(handles=legend_handles, loc="upper right")
418 fig.tight_layout()

```

```

419     fig.savefig(output_path, dpi=300, bbox_inches="tight")
420     plt.close(fig)
421
422
423     def write_typical_policy_doc(
424         doc_path: Path,
425         policy_summary: pd.DataFrame,
426         critical_soh_low: float,
427         critical_soh_high: float,
428     ) -> None:
429         long_policies = policy_summary.loc[policy_summary["lifetime_label"] == "长寿命策略"]
430         short_policies = policy_summary.loc[policy_summary["lifetime_label"] == "短寿命策略"]
431         longest = policy_summary.loc[policy_summary["longest_strategy"] == "是"].iloc[0]
432         shortest = policy_summary.loc[policy_summary["shortest_strategy"] == "是"].iloc[0]
433
434         lines = [
435             "# 典型长寿命与短寿命充电策略",
436             "",
437             "本分析使用第 200 次循环的 SOH 作为早期使用寿命代理指标。",
438             f"长寿命阈值为样本 SOH 的 75% 分位数 ({critical_soh_high:.6f})，",
439             f"短寿命阈值为 25% 分位数 ({critical_soh_low:.6f})。策略类型按策略内 SOH 中位数判定。",
440             "",
441             "## 关键结论",
442             "",
443             f"- **寿命最长策略: `{longest['policy']}`**, SOH 中位数 {longest['SOH_median']:.6f}, 范围 {longest['SOH_range']}。",
444             f"- **寿命最短策略: `{shortest['policy']}`**, SOH 中位数 {shortest['SOH_median']:.6f}, 范围 {shortest['SOH_range']}。",
445             "",
446             "## 典型长寿命策略",
447             "",
448             "| 策略 | 电池数 | SOH中位数 | SOH范围 | 平均充电时间范围/min | 标记 |",
449             "|---|---:|---:|---|---|",
450         ]
451         for _, row in long_policies.sort_values("SOH_median", ascending=False).iterrows():
452             mark = "寿命最长" if row["longest_strategy"] == "是" else "典型长寿命"
453             lines.append(
454                 f"| `{row['policy']}` | {int(row['battery_count'])} | {row['

```

```

        SOH_median']:.6f} | "
455     f"{row['SOH_range']} | {row['charge_time_range']} | **{mark}** | "
456 )
457 lines.extend(
458     [
459         "",
460         "## 典型短寿命策略",
461         "",
462         "| 策略 | 电池数 | SOH中位数 | SOH范围 | 平均充电时间范围/min | 标记 | "
463         ,
464         "|---|---:|---:|---|---|---|",
465     ]
466 )
467 for _, row in short_policies.sort_values("SOH_median").iterrows():
468     mark = "寿命最短" if row["shortest_strategy"] == "是" else "典型短寿命"
469     lines.append(
470         f"| `{row['policy']}` | {int(row['battery_count'])} | {row['SOH_median']:.6f} | "
471         f"{row['SOH_range']} | {row['charge_time_range']} | **{mark}** | "
472     )
473 lines.extend(
474     [
475         "",
476         "## 解释限制",
477         "",
478         "- 各策略的完整 200 循环样本数较少，结论用于本数据集内的典型策略比较，不宜直接外推为因果结论。",
479         "- `4_8C_80PER_4_8C_NEWSTRUCTURE` 的中位数较高但包含一个明显较低的 SOH 个体，需结合离散程度解读。",
480         "- 数据未覆盖 SOH<80% 的真实寿命终止循环，因此这里比较的是第 200 次循环的健康保持程度。",
481     ]
482 )
483 doc_path.parent.mkdir(parents=True, exist_ok=True)
484 doc_path.write_text("\n".join(lines) + "\n", encoding="utf-8")
485
486 def write_policy_report(
487     report_path: Path,
488     policy_summary: pd.DataFrame,
489     critical_soh_low: float,
490     critical_soh_high: float,
491 ) -> None:
492     """输出第三次迭代要求的统一策略统计报告。"""

```

```

493 lines = [
494     "# 统一充电策略统计报告",
495     "",
496     "## 统计口径",
497     "",
498     "- 策略名称已按第三次迭代规则合并，并移除 `_NEWSTRUCTURE` 后缀。",
499     "- 最终 SOH 为完整记录至第 200 次循环的电池在 `cycle=200` 时的清洗后 SOH。"
500     ,
501     "- 充电时间、内阻和温度先按电池对 1 - 200 次循环取均值，再按策略汇总。",
502     f"- 下四分位线 Q1={critical_soh_low:.6f}; 上四分位线 Q3={critical_soh_high:.6f}。",
503     "",
504     "## 最终 SOH 与充电时间",
505     "",
506     "| 统一策略 | 电池记号 | 电池数 | 最终SOH平均 | 最小 | 最大 | 充电时间平均/min | 最小/min | 最大/min |",
507     "|---|---|---:|---:|---:|---:|---:|---:|",
508 ]
509 for _, row in policy_summary.iterrows():
510     lines.append(
511         f"| `{row['policy']}` | {row['battery_ids']} | {int(row['battery_count'])} | "
512         f"{row['SOH_mean']:.6f} | {row['SOH_min']:.6f} | {row['SOH_max']:.6f} | "
513         f"{row['charge_time_mean']:.6f} | {row['charge_time_min']:.6f} | {row['charge_time_max']:.6f} | "
514     )
515 lines.extend(
516     [
517         "",
518         "## 其他参数",
519         "",
520         "| 统一策略 | C1取值 | Q1取值 | C2取值 | 初始容量平均/Ah | 最小/Ah | 最大/Ah | 平均内阻 | 平均温度/°C |",
521         "|---|---|---|---|---:|---:|---:|---:|",
522     ]
523 )
524 for _, row in policy_summary.iterrows():
525     lines.append(
526         f"| `{row['policy']}` | {row.get('C1_values', '')} | {row.get('Q1_values', '')} | "
527         f"{row.get('C2_values', '')} | {row['initial_capacity_mean']:.6f} | "
528         f"{row['initial_capacity_min']:.6f} | {row['initial_capacity_max']:.6f} | "
529     )

```

```

528         f"{row['IR_mean']:.6f} | {row['Tavg_mean']:.3f} | "
529     )
530     lines.extend(
531         [
532             "",
533             "## 解释限制",
534             "",
535             "- 9块仅有150次循环数据的测试电池未纳入最终SOH策略统计。",
536             "- 统一策略是按题目指定规则进行的分析分组；报告中的相关性不代表充电策略对
537               寿命的因果效应。",
538             "- 本数据片段尚未覆盖 SOH<80% 的真实寿命终止点，最终SOH用于比较第200次循
539               环时的健康保持程度。",
540         ]
541     )
542     report_path.parent.mkdir(parents=True, exist_ok=True)
543     report_path.write_text("\n".join(lines) + "\n", encoding="utf-8")
544
545 def build_acceptance_results(
546     original_summary: pd.DataFrame,
547     normalized_summary: pd.DataFrame,
548     normalized_cycles: pd.DataFrame,
549     policy_table: pd.DataFrame,
550     cycle_200: pd.DataFrame,
551     policy_summary: pd.DataFrame,
552     soh_dir: Path,
553     table_write_paths: dict[str, Path],
554 ) -> list[tuple[str, bool, str]]:
555     """执行第三次迭代的显式验收检查。"""
556     original_count = int(original_summary["policy"].nunique())
557     normalized_count = int(normalized_summary["policy"].nunique())
558     expected_names = {normalize_policy_name(value) for value in original_summary
559                       ["policy"].unique()}
560     actual_names = set(normalized_summary["policy"].unique())
561     forbidden = {"80PER_3_6C", "4_8C_80PER_4_8C_NEWSTRUCTURE"}
562     plot_files = list(soh_dir.glob("SOH_*.png"))
563     individual_files = [path for path in plot_files if path.stem != "SOH_ALL"]
564     results = [
565         (
566             "策略数量恰好减少2种",
567             original_count - normalized_count == 2,
568             f"原始{original_count}种，统一后{normalized_count}种，减少{
569               original_count - normalized_count}种",
570         ),

```

```

568     (
569         "统一策略集合正确",
570         actual_names == expected_names,
571         f"实际策略: {sorted(actual_names)}",
572     ),
573     (
574         "旧策略名称已消除",
575         not (actual_names & forbidden),
576         f"禁止名称残留: {sorted(actual_names & forbidden)}",
577     ),
578     (
579         "NEWSTRUCTURE后缀已全部移除",
580         not any("NEWSTRUCTURE" in name.upper() for name in actual_names),
581         "统一后的策略名不含NEWSTRUCTURE",
582     ),
583     (
584         "全部处理表命名一致",
585         set(normalized_cycles["policy"].unique()) == actual_names
586         and set(policy_table["policy"].unique()) == actual_names
587         and set(cycle_200["policy"].unique()).issubset(actual_names),
588         "清洗循环表、charge_policy和battery_SOH_charge均采用统一名称",
589     ),
590     (
591         "charge_policy覆盖49块电池",
592         len(policy_table) == 49
593         and policy_table["battery_id"].nunique() == 49,
594         f"记录数={len(policy_table)}, 唯一电池数={policy_table['battery_id'].nunique()}",
595     ),
596     (
597         "核心CSV均写入标准文件名",
598         all(path.name == expected for expected, path in table_write_paths.items()),
599         ", ".join(f"{expected}->{path.name}" for expected, path in table_write_paths.items()),
600     ),
601     (
602         "策略报告覆盖全部统一策略",
603         set(policy_summary["policy"]) == actual_names and len(policy_summary) == normalized_count,
604         f"报告策略数={len(policy_summary)}",
605     ),
606     (
607         "49张单体曲线与SOH_ALL齐全",

```

```

608         len(individual_files) == 49 and (soh_dir / "SOH_ALL.png").is_file(),
609         f"单体曲线={len(individual_files)}, SOH_ALL={int((soh_dir / 'SOH_ALL.
        png').is_file())}",
610     ),
611 ]
612 return results
613
614
615 def write_acceptance_report(
616     path: Path,
617     results: list[tuple[str, bool, str]],
618     mapping: dict[str, str],
619 ) -> None:
620     lines = [
621         "# 第三次迭代验收确认",
622         "",
623         "| 验收项 | 结果 | 证据 |",
624         "|---|---|---|",
625     ]
626     for name, passed, evidence in results:
627         lines.append(f"| {name} | {'通过' if passed else '失败'} | {evidence} |")
628     lines.extend(["", "## 策略名称映射", "", "| 原始名称 | 统一名称 |", "
        |---|---|"])
629     for source, target in mapping.items():
630         lines.append(f"| `{source}` | `{target}` |")
631     lines.extend(["", f"**总体结论: {'全部通过' if all(item[1] for item in
        results) else '存在失败项'}.**"])
632     path.parent.mkdir(parents=True, exist_ok=True)
633     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
634
635
636 def write_csv_if_changed(data: pd.DataFrame, path: Path) -> Path:
637     """内容不变时避免重写，兼容 CSV 正被表格软件打开的情形。"""
638     if path.exists():
639         try:
640             existing = pd.read_csv(path)
641             pd.testing.assert_frame_equal(existing, data, check_dtype=False)
642             return path
643         except (AssertionError, OSError, ValueError):
644             pass
645     pending_path = path.with_name(f"{path.stem}_v3_pending{path.suffix}")
646     try:
647         data.to_csv(path, index=False, encoding="utf-8-sig")
648         if pending_path.exists():

```

```

649         pending_path.unlink()
650     return path
651 except PermissionError:
652     data.to_csv(pending_path, index=False, encoding="utf-8-sig")
653     return pending_path
654
655
656 def write_log(
657     log_path: Path,
658     summary: pd.DataFrame,
659     cycles: pd.DataFrame,
660     audit: list[dict[str, float | int | str]],
661     cycle_200: pd.DataFrame,
662     policy_summary: pd.DataFrame,
663     critical_soh_low: float,
664     critical_soh_high: float,
665     iqr_factor: float,
666     font_name: str,
667     output_paths: list[Path],
668     prompt_text: str,
669     policy_mapping: dict[str, str],
670     acceptance_results: list[tuple[str, bool, str]],
671 ) -> None:
672     counts = Counter(str(item["column"]) for item in audit)
673     max_cycle_counts = cycles.groupby("battery_id")["cycle"].max().value_counts
674         ().sort_index()
675     missing_summary = int(summary.isna().sum().sum())
676     missing_cycles = int(cycles.isna().sum().sum())
677     long_count = int((cycle_200["SOH"] >= critical_soh_high).sum())
678     short_count = int((cycle_200["SOH"] <= critical_soh_low).sum())
679     middle_count = len(cycle_200) - long_count - short_count
680     longest = policy_summary.loc[policy_summary["longest_strategy"] == "是", "
        policy"].iloc[0]
681     shortest = policy_summary.loc[policy_summary["shortest_strategy"] == "是", "
        policy"].iloc[0]
682     normalized_policy_count = len(set(policy_mapping.values()))
683
684     lines = [
685         "# 问题一第三次迭代运行日志 (log_3) ",
686         "",
687         f"- 运行时间: {datetime.now().astimezone().isoformat(timespec='seconds')}
        ",
688         f"- IQR 系数: {iqr_factor:.6g}",
689         f"- 上四分位线 Q3 (75%分位数): {critical_soh_high:.6f}",

```



```

689 f"- 下四分位线 Q1 (25%分位数) : {critical_soh_low:.6f}",
690 f"- 绘图字体: {font_name}",
691 "",
692 "## 输入质量检查",
693 "",
694 f"- battery_summary.csv: {len(summary)} 行, {summary.shape[1]} 列, {
    summary['battery_id'].nunique()} 块电池。",
695 f"- cycle_train.csv: {len(cycles)} 行, {cycles.shape[1]} 列。",
696 f"- battery_id + cycle 重复记录: {int(cycles.duplicated(['battery_id', '
    cycle']).sum())}。",
697 f"- 汇总表缺失单元格: {missing_summary} (其中 C1 的结构性缺失保留, 不参与循
    环数据清洗)。",
698 f"- 循环表缺失单元格: {missing_cycles}。",
699 f"- 最大循环数分布: {dict((int(k), int(v)) for k, v in max_cycle_counts.
    items())}。",
700 "",
701 "## 策略统一命名",
702 "",
703 f"- 原始策略数: {len(policy_mapping)}; 统一后策略数: {
    normalized_policy_count}; 恰好减少 {len(policy_mapping) -
    normalized_policy_count} 种。",
704 "- `80PER_3_6C` 统一为 `3_6C-80PER_3_6C`。",
705 "- `4_8C_80PER_4_8C_NEWSTRUCTURE` 统一为 `4_8C_80PER_4_8C`。",
706 "- 其余策略仅移除 `_NEWSTRUCTURE` 后缀。",
707 "",
708 "## IQR 清洗",
709 "",
710 "- 方法: 按 battery_id 分组, 对 capacity、SOH、SOH_smooth、chargetime、IR
    、Tavg 分别计算 Q1、Q3 和 IQR。",
711 "- 判定: 小于 Q1 - 1.5×IQR 或大于 Q3 + 1.5×IQR 的值标记为异常。",
712 "- 填补: 使用同一电池、同一字段中距离最近的前后正常值均值; 边界点仅有一侧正
    常值时使用该侧值。",
713 f"- 共替换 {len(audit)} 个单元格; 分字段数量: {dict(counts)}。",
714 "- 原始 data 文件未改写; 清洗结果另存为 output/A/cycle_train_cleaned.csv。
    ",
715 "",
716 "## 200 次循环样本",
717 "",
718 f"- 纳入: 最大 cycle 恰为 200 的 {len(cycle_200)} 块电池。",
719 f"- 排除: 仅记录至 150 次循环的 {summary['prediction_test'].sum()} 块测试
    电池。",
720 f"- SOH 范围: {cycle_200['SOH'].min():.6f} - {cycle_200['SOH'].max():.6f}
    , 均值 {cycle_200['SOH'].mean():.6f}。",
721 f"- 按两个分位数分类: 长寿命 {long_count} 块, 中等寿命 {middle_count} 块,

```

```

短寿命 {short_count} 块。",
722 f"- 寿命最长策略（按策略 SOH 中位数）: {longest}。",
723 f"- 寿命最短策略（按策略 SOH 中位数）: {shortest}。",
724 "",
725 "## SOH 曲线",
726 "",
727 "- 使用清洗后的 `SOH_smooth` 绘制49张单体曲线和1张全部电池曲线。",
728 "- 单体曲线命名为 `SOH_N.png`; 总图命名为 `SOH_ALL.png`, 颜色按电池编号由蓝
    到红渐变。",
729 "",
730 "## 验收确认",
731 "",
732 ]
733 for name, passed, evidence in acceptance_results:
734     lines.append(f"- [{ 'x' if passed else ' '}] {name}: {evidence}。")
735 lines.extend(["", "## 输出文件", ""])
736 lines.extend(f"- `{path.relative_to(PROJECT_ROOT).as_posix()}` for path in
    output_paths")
737 lines.extend(
738     [
739         "",
740         "## 说明",
741         "",
742         "- 箱型图纵轴采用“第 200 次循环 SOH”作为使用寿命代理指标; 本题数据尚未
            覆盖 SOH<80% 的真实寿命终止循环。",
743         "- IQR 是统计异常筛查, 不等同于确认传感器故障; 清洗结果用于完成本题指定
            分析。",
744         "- 图中的两条参考线统一命名为上四分位线 (Q3) 和下四分位线 (Q1)。",
745         "- 策略报告使用统一后的7种策略, 最终SOH统计只纳入完整200循环的40块电池。
            ",
746         "",
747         "## 原始提示词",
748         "",
749         "以下完整保留本次执行时 `A/问题一/AGENTS.md` 的内容: ",
750         "",
751         "```text",
752         prompt_text.rstrip(),
753         "```",
754     ]
755 )
756 log_path.parent.mkdir(parents=True, exist_ok=True)
757 log_path.write_text("\n".join(lines) + "\n", encoding="utf-8")
758
759

```

```

760 def parse_args() -> argparse.Namespace:
761     parser = argparse.ArgumentParser(description="运行问题一完整分析流程")
762     parser.add_argument("--iqr-factor", type=float, default=1.5, help="IQR 异常
763         判定系数, 默认 1.5")
764
765     return parser.parse_args()
766
767 def main() -> None:
768     args = parse_args()
769     if args.iqr_factor <= 0:
770         raise ValueError("--iqr-factor 必须大于 0")
771
772     data_dir = PROJECT_ROOT / "data"
773     output_dir = PROJECT_ROOT / "output" / "A"
774     output_dir.mkdir(parents=True, exist_ok=True)
775     original_summary = pd.read_csv(data_dir / "battery_summary.csv")
776     original_cycles = pd.read_csv(data_dir / "cycle_train.csv")
777     validate_inputs(original_summary, original_cycles)
778     summary, cycles, policy_mapping = normalize_policies(original_summary,
779         original_cycles)
780     validate_inputs(summary, cycles)
781
782     cleaned, audit = iqr_clean_by_battery(cycles, MEASUREMENT_COLUMNS, args.
783         iqr_factor)
784     policy_table = build_policy_table(summary)
785     cycle_200 = build_cycle_200_summary(cleaned, policy_table)
786     critical_soh_low = float(cycle_200["SOH"].quantile(0.25))
787     critical_soh_high = float(cycle_200["SOH"].quantile(0.75))
788     policy_order = list(dict.fromkeys(summary.sort_values("battery_id")["policy"]
789         ))
790
791     policy_summary = build_policy_summary(
792         cycle_200,
793         policy_order,
794         critical_soh_low,
795         critical_soh_high,
796         summary,
797         cleaned,
798     )
799
800     cleaned_path = output_dir / "cycle_train_cleaned.csv"
801     policy_path = output_dir / "charge_policy.csv"
802     summary_path = output_dir / "battery_SOH_charge.csv"
803     soh_example_path = output_dir / "SOH_1_example.png"
804     histogram_path = output_dir / "SOH_summary.png"

```

```

800 scatter_path = output_dir / "SOH_chargetime.png"
801 boxplot_path = output_dir / "charge_policy.png"
802 policy_summary_path = output_dir / "policy.csv"
803 report_path = PROJECT_ROOT / "output" / "doc" / "1.md"
804 acceptance_path = output_dir / "acceptance_3.md"
805 soh_dir = PROJECT_ROOT / "A" / "问题一" / "SOH"
806
807 cleaned_write_path = write_csv_if_changed(cleaned, cleaned_path)
808 policy_write_path = write_csv_if_changed(policy_table, policy_path)
809 summary_write_path = write_csv_if_changed(cycle_200, summary_path)
810 policy_summary_write_path = write_csv_if_changed(policy_summary,
811         policy_summary_path)
812
812 font_name = configure_chinese_font()
813 plot_battery_soh(cleaned, 1, soh_example_path)
814 save_soh_histogram(cycle_200, histogram_path, critical_soh_low,
815         critical_soh_high)
816 save_soh_charge_scatter(cycle_200, scatter_path, policy_order)
817 save_policy_boxplot(
818     cycle_200,
819     boxplot_path,
820     critical_soh_low,
821     critical_soh_high,
822     policy_order,
823 )
824 write_policy_report(
825     report_path,
826     policy_summary,
827     critical_soh_low,
828     critical_soh_high,
829 )
830
830 soh_dir.mkdir(parents=True, exist_ok=True)
831 for stale_plot in soh_dir.glob("SOH*.png"):
832     stale_plot.unlink()
833 for battery_id in sorted(int(value) for value in cleaned["battery_id"].
834     unique()):
835     plot_battery_soh(cleaned, battery_id, soh_dir / f"SOH_{battery_id}.png")
836 plot_all_battery_soh(cleaned, soh_dir / "SOH_ALL.png")
837
837 acceptance_results = build_acceptance_results(
838     original_summary,
839     summary,
840     cleaned,

```

```

841     policy_table,
842     cycle_200,
843     policy_summary,
844     soh_dir,
845     {
846         cleaned_path.name: cleaned_write_path,
847         policy_path.name: policy_write_path,
848         summary_path.name: summary_write_path,
849         policy_summary_path.name: policy_summary_write_path,
850     },
851 )
852 write_acceptance_report(acceptance_path, acceptance_results, policy_mapping)
853
854 output_paths = [
855     cleaned_write_path,
856     policy_write_path,
857     summary_write_path,
858     soh_example_path,
859     histogram_path,
860     scatter_path,
861     boxplot_path,
862     policy_summary_write_path,
863     report_path,
864     acceptance_path,
865     soh_dir,
866 ]
867 write_log(
868     PROJECT_ROOT / "A" / "问题一" / "log_3.md",
869     summary,
870     cycles,
871     audit,
872     cycle_200,
873     policy_summary,
874     critical_soh_low,
875     critical_soh_high,
876     args.iqr_factor,
877     font_name,
878     output_paths,
879     (PROJECT_ROOT / "A" / "问题一" / "AGENTS.md").read_text(encoding="utf-8")
880     ,
881     policy_mapping,
882     acceptance_results,
883 )

```

```

884     print(f"IQR 替换单元格数: {len(audit)}")
885     print(f"200 次循环样本数: {len(cycle_200)}")
886     print(f"critical_SOH_low: {critical_soh_low:.6f}")
887     print(f"critical_SOH_high: {critical_soh_high:.6f}")
888     print(f"策略数量: {len(policy_mapping)} -> {len(set(policy_mapping.values()))}")
889     print(f"SOH曲线数量: {len(list(soh_dir.glob('SOH_*.png')))}")
890     failed = [name for name, passed, _ in acceptance_results if not passed]
891     print(f"验收失败项: {failed}")
892     for path in output_paths:
893         print(f"已生成: {path}")
894
895
896 if __name__ == "__main__":
897     main()

```

B.3 task2_statistical_analysis.py

Listing 3 问题二统计分析脚本

```

1  """问题二：两阶段快充参数与 200 圈健康保持程度的统计分析。
2
3  运行: python src/A/task2_statistical_analysis.py
4
5  说明: 赛题 CSV 未观测 SOH=0.8 的 EOL, 因此本脚本仅分析 SOH_200、
6  200 圈容量保持率和早期衰减斜率; 不将任何外推结果称为真实循环寿命。
7  """
8
9  from __future__ import annotations
10
11 from itertools import combinations
12 from pathlib import Path
13
14 import matplotlib.pyplot as plt
15 import numpy as np
16 import pandas as pd
17 import seaborn as sns
18 from sklearn.cross_decomposition import PLSRegression
19 from sklearn.linear_model import RidgeCV
20 from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
21 from sklearn.model_selection import LeaveOneOut
22 from sklearn.pipeline import make_pipeline
23 from sklearn.preprocessing import StandardScaler
24 import statsmodels.formula.api as smf

```

```

25 from scipy import stats
26 from statsmodels.stats.multitest import multipletests
27 from statsmodels.stats.outliers_influence import variance_inflation_factor
28
29 from SOH_plot import configure_chinese_font
30
31
32 PROJECT_ROOT = Path(__file__).resolve().parents[2]
33 DATA_DIR = PROJECT_ROOT / "data"
34 OUTPUT_DIR = PROJECT_ROOT / "output" / "A" / "question2"
35 FIG_DIR = OUTPUT_DIR / "figures"
36
37 POLICY_ORDER = [
38     "4_8C_80PER_4_8C",
39     "5C_67PER_4C",
40     "5_3C_54PER_4C",
41     "5_6C_19PER_4_6C",
42     "5_6C_36PER_4_3C",
43     "3_7C_31PER_5_9C",
44 ]
45
46
47 def canonical_policy(value: str) -> str:
48     """统一问题一已采用的策略命名规则。"""
49     value = str(value).replace("_NEWSTRUCTURE", "")
50     if value == "80PER_3_6C":
51         return "3_6C-80PER_3_6C"
52     return value
53
54
55 def holm_pairwise(data: pd.DataFrame) -> pd.DataFrame:
56     rows = []
57     policies = [x for x in POLICY_ORDER if x in set(data["policy"])]
58     for left, right in combinations(policies, 2):
59         x = data.loc[data["policy"] == left, "SOH_200"]
60         y = data.loc[data["policy"] == right, "SOH_200"]
61         result = stats.mannwhitneyu(x, y, alternative="two-sided", method="auto")
62         rows.append(
63             {
64                 "policy_left": left,
65                 "policy_right": right,
66                 "n_left": len(x),
67                 "n_right": len(y),
68                 "median_difference_left_minus_right": float(x.median() - y.median())

```

```

        ()),
        "U": float(result.statistic),
        "p_raw": float(result.pvalue),
    }
)
result = pd.DataFrame(rows)
if not result.empty:
    result["p_holm"] = multipletests(result["p_raw"], method="holm")[1]
return result

def policy_summary(data: pd.DataFrame) -> pd.DataFrame:
    return (
        data.groupby("policy", observed=True)
        .agg(
            battery_count=("battery_id", "size"),
            SOH_200_mean=("SOH_200", "mean"),
            SOH_200_median=("SOH_200", "median"),
            SOH_200_std=("SOH_200", "std"),
            SOH_200_min=("SOH_200", "min"),
            SOH_200_max=("SOH_200", "max"),
            fade_200_mean=("fade_200", "mean"),
            fade_slope_mean=("fade_slope", "mean"),
            charge_time_mean=("charge_time_mean", "mean"),
        )
        .reindex([p for p in POLICY_ORDER if p in set(data["policy"])])
        .reset_index()
    )

def extract_cell_metrics(summary: pd.DataFrame, cycles: pd.DataFrame) -> tuple[
    pd.DataFrame, pd.DataFrame]:
    summary = summary.copy()
    cycles = cycles.copy()
    summary["policy"] = summary["policy"].map(canonical_policy)
    cycles["policy"] = cycles["policy"].map(canonical_policy)

    max_cycle = cycles.groupby("battery_id")["cycle"].max().rename("max_cycle")
    cell = summary.merge(max_cycle, on="battery_id", validate="one_to_one")
    cell = cell.query("prediction_test == 0 and max_cycle == 200").copy()

    cycle_200 = cycles.loc[cycles["cycle"] == 200, ["battery_id", "SOH", "
        SOH_smooth"]].copy()
    cycle_200 = cycle_200.rename(columns={"SOH": "SOH_200_raw", "SOH_smooth": "

```



```

    SOH_200"}})
110 cycle_1 = cycles.loc[cycles["cycle"] == 1, ["battery_id", "SOH_smooth"]].
    rename(
111     columns={"SOH_smooth": "SOH_1"}
112 )
113 cell = cell.merge(cycle_1, on="battery_id", validate="one_to_one").merge(
114     cycle_200, on="battery_id", validate="one_to_one"
115 )
116 cell["fade_200"] = cell["SOH_1"] - cell["SOH_200"]
117 cell["fade_slope"] = cell["fade_200"] / 199.0
118 cell["charge_time_mean"] = cell["mean_chargetime"]
119
120 # 主分析：同一实验结构（dataset 3）；全体完整样本仅作敏感性对照。
121 primary = cell.query("dataset_id == 3").copy()
122 longitudinal = cycles.merge(
123     primary[["battery_id", "policy"]], on=["battery_id", "policy"], how="
        inner", validate="many_to_one"
124 ).copy()
125 longitudinal["cycle_scaled"] = longitudinal["cycle"] / 100.0
126 return cell, primary, longitudinal
127
128
129 def save_figures(primary: pd.DataFrame, longitudinal: pd.DataFrame, model) ->
    None:
130     sns.set_theme(style="whitegrid", context="notebook")
131     configure_chinese_font()
132
133     order = [p for p in POLICY_ORDER if p in set(primary["policy"])]
134     fig, ax = plt.subplots(figsize=(13, 6))
135     sns.boxplot(data=primary, x="policy", y="SOH_200", order=order, ax=ax, color
        ="#9ecae1", fliersize=0)
136     sns.stripplot(data=primary, x="policy", y="SOH_200", order=order, ax=ax,
        color="#1f1f1f", size=5, jitter=0.15)
137     ax.set(title="同一实验结构下各策略的第 200 圈 SOH", xlabel="两阶段快充策略",
        ylabel="SOH at cycle 200")
138     ax.tick_params(axis="x", rotation=22)
139     fig.tight_layout()
140     fig.savefig(FIG_DIR / "policy_soh200_boxplot.png", dpi=300)
141     plt.close(fig)
142
143     fig, ax = plt.subplots(figsize=(11, 6))
144     for policy in order:
145         group = longitudinal.loc[longitudinal["policy"] == policy]
146         agg = group.groupby("cycle", as_index=False)["SOH_smooth"].agg(["mean", "

```

```

    sem"])).reset_index()
147 x = agg["cycle"].to_numpy()
148 y = agg["mean"].to_numpy()
149 sem = agg["sem"].fillna(0).to_numpy()
150 ax.plot(x, y, linewidth=2, label=policy)
151 ax.fill_between(x, y - 1.96 * sem, y + 1.96 * sem, alpha=0.12)
152 ax.set(title="同一实验结构下的 SOH 轨迹 (均值  $\pm$  95% 近似区间)", xlabel="Cycle",
    ylabel="Smoothed SOH")
153 ax.legend(fontsize=8, ncol=2)
154 fig.tight_layout()
155 fig.savefig(FIG_DIR / "policy_soh_trajectories.png", dpi=300)
156 plt.close(fig)
157
158 residuals = model.resid
159 fitted = model.fittedvalues
160 fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))
161 axes[0].scatter(fitted, residuals, color="#3182bd", alpha=0.8)
162 axes[0].axhline(0, color="black", linewidth=1)
163 axes[0].set(title="参数回归残差图", xlabel="Fitted fade at cycle 200", ylabel="Residual")
164 stats.probplot(residuals, dist="norm", plot=axes[1])
165 axes[1].set_title("参数回归 Q-Q 图")
166 fig.tight_layout()
167 fig.savefig(FIG_DIR / "parameter_regression_diagnostics.png", dpi=300)
168 plt.close(fig)
169
170
171 def nested_loo_models(primary: pd.DataFrame) -> tuple[pd.DataFrame, pd.
    DataFrame]:
172     """Compare collinearity-resistant models with battery-level nested LOOCV."""
173     features = ["C1", "Q1", "C2"]
174     X = primary[features].to_numpy(dtype=float)
175     y = primary["fade_200"].to_numpy(dtype=float)
176     outer = LeaveOneOut()
177     predictions = {name: np.zeros(len(primary)) for name in ("OLS", "Ridge", "
        PLS_1component", "PLS_2components")}
178     alphas = np.logspace(-6, 3, 80)
179     for train_idx, test_idx in outer.split(X):
180         X_train, X_test = X[train_idx], X[test_idx]
181         y_train = y[train_idx]
182         # Scaling and penalty selection occur inside every training fold.
183         models = {
184             "OLS": make_pipeline(StandardScaler(), RidgeCV(alphas=[1e-12], cv=
                LeaveOneOut(), scoring="neg_mean_squared_error")),

```

```

185         "Ridge": make_pipeline(StandardScaler(), RidgeCV(alphas=alphas, cv=
            LeaveOneOut(), scoring="neg_mean_squared_error")),
186         "PLS_1component": make_pipeline(StandardScaler(), PLSRegression(
            n_components=1)),
187         "PLS_2components": make_pipeline(StandardScaler(), PLSRegression(
            n_components=2)),
188     }
189     for name, fitted_model in models.items():
190         fitted_model.fit(X_train, y_train)
191         predictions[name][test_idx[0]] = float(fitted_model.predict(X_test).
            ravel()[0])
192     comparison = pd.DataFrame([
193         {"model": name, "LOOCV_MAE": mean_absolute_error(y, pred), "LOOCV_RMSE":
            mean_squared_error(y, pred) ** 0.5, "LOOCV_R2": r2_score(y, pred)}
194         for name, pred in predictions.items()
195     ]).sort_values("LOOCV_RMSE").reset_index(drop=True)
196     prediction_df = primary[["battery_id", "policy", "fade_200"]].copy()
197     for name, pred in predictions.items():
198         prediction_df[f"pred_{name}"] = pred
199     return comparison, prediction_df
200
201
202 def write_report(
203     cell: pd.DataFrame,
204     primary: pd.DataFrame,
205     summary: pd.DataFrame,
206     kruskal: stats.KruskalResult,
207     pairwise: pd.DataFrame,
208     correlations: pd.DataFrame,
209     regression,
210     vif: pd.DataFrame,
211     model_comparison: pd.DataFrame,
212     mixed_note: str,
213 ) -> None:
214     significant = pairwise.loc[pairwise["p_holm"] < 0.05] if not pairwise.empty
215     else pairwise
216     lines = [
217         "# 问题二统计分析报告",
218         "",
219         "## 结论口径",
220         "",
221         "- 赛题数据未观测 SOH=0.8 的 EOL; 本报告的响应变量是第 200 次循环 SOH (
            SOH_200) 及其对应衰减量, 不是实际循环寿命。",
222         "- 主分析仅保留 dataset_id=3、prediction_test=0 且完整记录至 200 圈的电

```

```

池，以降低跨批次和实验结构混杂；全体 40 个完整样本只作敏感性参考。",
222 f"- 全体可用完整样本: {len(cell)}; 主分析样本: {len(primary)}。",
223 "",
224 "## 策略差异检验",
225 "",
226 f"- Kruskal--Wallis: H={kruskal.statistic:.4f}, p={kruskal.pvalue:.4g}。
该检验比较各策略 SOH_200 分布是否一致。",
227 f"- Holm 校正后显著的两两比较数: {len(significant)}。",
228 "- 分组样本量有限且参数非正交，因此 p 值用于描述样本内证据，不作严格因果结
论。",
229 "",
230 "## 参数回归与相关",
231 "",
232 "- 回归响应: fade_200 = SOH_1 - SOH_200; 解释变量: C1、Q1、C2; 标准误使用
HC3 稳健协方差。",
233 f"- OLS adjusted R2={regression.rsquared_adj:.4f}, overall F-test p={
regression.f_pvalue:.4g}。",
234 "- 回归和相关性均为探索性结果，因为同一策略参数组合存在重复测量，而可区分策
略组合数有限。",
235 "",
236 "## 纵向模型",
237 "",
238 "- 使用 1--200 圈 SOH_smooth 拟合“cycle × policy”的线性混合效应模型，电
池设为随机截距和随机循环斜率。",
239 f"- {mixed_note}",
240 "",
241 "## 结果文件",
242 "",
243 "- cell_metrics.csv: 逐电池的 SOH_200、200 圈衰减量与策略参数。",
244 "- policy_summary.csv: 策略汇总。",
245 "- group_test.csv / pairwise_mannwhitney_holm.csv: 组间检验。",
246 "- correlations.csv / regression_coefficients.csv / vif.csv: 参数关联、回
归与共线性诊断。",
247 "- figures/: 箱线图、SOH 轨迹与回归诊断图。",
248 "",
249 "## 策略汇总",
250 "",
251 summary.round(6).to_markdown(index=False),
252 "",
253 "## Spearman 相关",
254 "",
255 correlations.round(6).to_markdown(index=False),
256 "",
257 "## OLS 系数 (HC3)",

```

```

258     "",
259     pd.DataFrame({"term": regression.params.index, "coefficient": regression.
        params.values, "p_value_hc3": regression.pvalues.values}).round(6).
        to_markdown(index=False),
260     "",
261     "## VIF",
262     "",
263     vif.round(6).to_markdown(index=False),
264 ]
265 (OUTPUT_DIR / "report.md").write_text("\n".join(lines) + "\n", encoding="utf
    -8")
266
267
268 def main() -> None:
269     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
270     FIG_DIR.mkdir(parents=True, exist_ok=True)
271     summary_raw = pd.read_csv(DATA_DIR / "battery_summary.csv")
272     cycles_raw = pd.read_csv(DATA_DIR / "cycle_train.csv")
273     cell, primary, longitudinal = extract_cell_metrics(summary_raw, cycles_raw)
274     cell.to_csv(OUTPUT_DIR / "cell_metrics.csv", index=False, encoding="utf-8-
        sig")
275     primary.to_csv(OUTPUT_DIR / "primary_cohort_cell_metrics.csv", index=False,
        encoding="utf-8-sig")
276
277     summary = policy_summary(primary)
278     summary.to_csv(OUTPUT_DIR / "policy_summary.csv", index=False, encoding="utf
        -8-sig")
279     groups = [primary.loc[primary["policy"] == p, "SOH_200"] for p in summary["
        policy"]]
280     kruskal = stats.kruskal(*groups)
281     epsilon_squared = max(0.0, (kruskal.statistic - len(groups) + 1) / (len(
        primary) - len(groups)))
282     pd.DataFrame([{"test": "Kruskal-Wallis", "H": kruskal.statistic, "p_value":
        kruskal.pvalue, "epsilon_squared": epsilon_squared, "n": len(primary)}])
        .to_csv(
283         OUTPUT_DIR / "group_test.csv", index=False, encoding="utf-8-sig"
284     )
285     pairwise = holm_pairwise(primary)
286     pairwise.to_csv(OUTPUT_DIR / "pairwise_mannwhitney_holm.csv", index=False,
        encoding="utf-8-sig")
287
288     corr_rows = []
289     for parameter in ["C1", "Q1", "C2"]:
290         rho, p_value = stats.spearmanr(primary[parameter], primary["fade_200"])

```

```

291     corr_rows.append({"parameter": parameter, "spearman_rho_with_fade_200":
292         rho, "p_value": p_value})
293 correlations = pd.DataFrame(corr_rows)
294 correlations.to_csv(OUTPUT_DIR / "correlations.csv", index=False, encoding="
    utf-8-sig")
295
296 regression = smf.ols("fade_200 ~ C1 + Q1 + C2", data=primary).fit(cov_type="
    HC3")
297 regression.summary().as_text()
298 (OUTPUT_DIR / "parameter_regression_summary.txt").write_text(regression.
    summary().as_text(), encoding="utf-8")
299 pd.DataFrame({"term": regression.params.index, "coefficient": regression.
    params.values, "std_error_hc3": regression.bse.values, "p_value_hc3":
    regression.pvalues.values}).to_csv(
300     OUTPUT_DIR / "regression_coefficients.csv", index=False, encoding="utf-8-
    sig"
301 )
302 X = primary[["C1", "Q1", "C2"]].astype(float)
303 vif = pd.DataFrame({"variable": X.columns, "VIF": [variance_inflation_factor
    (X.values, i) for i in range(X.shape[1])]})
304 vif.to_csv(OUTPUT_DIR / "vif.csv", index=False, encoding="utf-8-sig")
305 model_comparison, loo_predictions = nested_loo_models(primary)
306 model_comparison.to_csv(OUTPUT_DIR / "alternative_model_loocv.csv", index=
    False, encoding="utf-8-sig")
307 loo_predictions.to_csv(OUTPUT_DIR / "alternative_model_loo_predictions.csv",
    index=False, encoding="utf-8-sig")
308
309 try:
310     mixed = smf.mixedlm("SOH_smooth ~ cycle_scaled * C(policy)", longitudinal
    , groups=longitudinal["battery_id"], re_formula="~cycle_scaled").fit(
    method="lbfgs", reml=False)
311     (OUTPUT_DIR / "mixed_effects_summary.txt").write_text(mixed.summary().
    as_text(), encoding="utf-8")
312     mixed_note = f"模型收敛: {bool(mixed.converged)}; AIC={mixed.aic:.3f}。完
    整估计见 mixed_effects_summary.txt。"
313 except Exception as exc: # 保证其余统计结果仍可复现。
314     mixed_note = f"混合效应模型未稳定收敛: {type(exc).__name__}: {exc}。不据此
    输出系数结论。"
315     (OUTPUT_DIR / "mixed_effects_summary.txt").write_text(mixed_note + "\n",
    encoding="utf-8")
316
317 save_figures(primary, longitudinal, regression)
318 write_report(cell, primary, summary, kruskal, pairwise, correlations,
    regression, vif, model_comparison, mixed_note)

```

```

318     print(f"主分析样本数: {len(primary)}")
319     print(f"Kruskal-Wallis H={kruskal.statistic:.4f}, p={kruskal.pvalue:.6g}")
320     print(f"输出目录: {OUTPUT_DIR}")
321
322
323 if __name__ == "__main__":
324     main()

```

Listing 4 问题二 EOL 寿命策略分析

```

1  """问题二：以公开同源 EOL 为响应的策略寿命分析。
2
3  数据边界：只使用 prediction_test=0 的公开 MATR 标签；不读取、训练、比较或输出
4  9 块问题三测试电池的 EOL。本脚本分析的是六个既有策略组合的关联，不能产生
5  单个 C1、Q1、C2 参数的严格因果效应。
6  """
7
8  from __future__ import annotations
9
10 from itertools import combinations
11 from pathlib import Path
12
13 import matplotlib
14
15 matplotlib.use("Agg")
16 import matplotlib.pyplot as plt
17 import numpy as np
18 import pandas as pd
19 from scipy import stats
20
21
22 ROOT = Path(__file__).resolve().parents[2]
23 SUMMARY = ROOT / "data" / "battery_summary.csv"
24 AUDIT = ROOT / "references" / "matr_source" / "external_eol_audit" / "
    contest_to_matr_eol_audit.csv"
25 OUT = ROOT / "A" / "问题二" / "output" / "eol_lifetime"
26 FIG = OUT / "figures"
27 Q_STAR = 50.0
28 SOC_END = 80.0
29
30
31 def canonical_policy(value: str) -> str:
32     value = str(value).replace("_NEWSTRUCTURE", "")
33     return "3_6C-80PER_3_6C" if value == "80PER_3_6C" else value
34

```

```

35
36 def exposure(c1: float, q1: float, c2: float) -> tuple[float, float, float]:
37     """固定 50% SOC 分界下的低/高 SOC 电流暴露。"""
38     if q1 >= Q_STAR:
39         low = c1 * Q_STAR
40         high = c1 * (q1 - Q_STAR) + c2 * (SOC_END - q1)
41     else:
42         low = c1 * q1 + c2 * (Q_STAR - q1)
43         high = c2 * (SOC_END - Q_STAR)
44     return float(low), float(high), float(high / (SOC_END - Q_STAR))
45
46
47 def holm_pairs(data: pd.DataFrame) -> pd.DataFrame:
48     rows = []
49     policies = list(data.policy.drop_duplicates())
50     for left, right in combinations(policies, 2):
51         x = data.loc[data.policy == left, "external_eol_cycles"]
52         y = data.loc[data.policy == right, "external_eol_cycles"]
53         test = stats.mannwhitneyu(x, y, alternative="two-sided", method="
54             asymptotic")
55         rows.append({"policy_a": left, "policy_b": right, "u": test.statistic, "
56             p_raw": test.pvalue})
57     out = pd.DataFrame(rows).sort_values("p_raw").reset_index(drop=True)
58     m = len(out)
59     adjusted = []
60     running = 0.0
61     for index, p in enumerate(out.p_raw):
62         running = max(running, min(1.0, (m - index) * p))
63         adjusted.append(running)
64     out["p_holm"] = adjusted
65     return out
66
67
68 def markdown_table(frame: pd.DataFrame, digits: int = 3) -> str:
69     """不依赖可选 tabulate 包的简易 Markdown 表格。"""
70     def display(value: object) -> str:
71         if isinstance(value, (float, np.floating)):
72             return f"{value:.{digits}f}"
73         return str(value)
74
75     headers = [str(c) for c in frame.columns]
76     lines = ["| " + " | ".join(headers) + " |", "| " + " | ".join(["---"] * len(
77         headers)) + " |"]
78     lines.extend("| " + " | ".join(display(v) for v in row) + " |" for row in

```



```

        frame.itertuples(index=False, name=None))
76 return "\n".join(lines)
77
78
79 def weighted_ols(data: pd.DataFrame, columns: list[str]) -> dict[str, object]:
80     """策略均值上的 WLS; 推断单位为策略配方而非重复循环记录。"""
81     x = np.column_stack([np.ones(len(data)), data[columns].to_numpy(float)])
82     y = data.log_eol_mean.to_numpy(float)
83     weights = data.n.to_numpy(float)
84     root_w = np.sqrt(weights)
85     beta = np.linalg.lstsq(x * root_w[:, None], y * root_w, rcond=None)[0]
86     fitted = x @ beta
87     resid = y - fitted
88     n, k = x.shape
89     ssr = float(np.sum(weights * resid**2))
90     sst = float(np.sum(weights * (y - np.average(y, weights=weights)) ** 2))
91     r2 = 1.0 - ssr / sst
92     adj_r2 = 1.0 - (1.0 - r2) * (n - 1) / (n - k)
93     # df=3, 只把区间作为模型内描述, 不把它解释为强假设检验。
94     sigma2 = ssr / max(n - k, 1)
95     covariance = sigma2 * np.linalg.inv((x * weights[:, None]).T @ x)
96     se = np.sqrt(np.diag(covariance))
97     t = beta / np.maximum(se, 1e-12)
98     p = 2 * stats.t.sf(abs(t), df=max(n - k, 1))
99     return {
100         "beta": beta,
101         "se": se,
102         "p": p,
103         "fitted": fitted,
104         "resid": resid,
105         "r2": r2,
106         "adj_r2": adj_r2,
107         "n": n,
108         "k": k,
109     }
110
111
112 def fit_rows(policy: pd.DataFrame, columns: list[str], label: str) -> tuple[pd.
    DataFrame, pd.DataFrame]:
113     fit = weighted_ols(policy, columns)
114     terms = ["Intercept", *columns]
115     coef = pd.DataFrame({
116         "model": label,
117         "term": terms,

```

```

118         "coefficient_log_eol": fit["beta"],
119         "std_error": fit["se"],
120         "p_classical_descriptive": fit["p"],
121     })
122     stats_row = pd.DataFrame([
123         "model": label, "n_policies": fit["n"], "parameters": fit["k"],
124         "r_squared": fit["r2"], "adj_r_squared": fit["adj_r2"],
125     ])
126     return coef, stats_row
127
128
129 def leave_one_policy(policy: pd.DataFrame, columns: list[str]) -> pd.DataFrame:
130     rows = []
131     for held in policy.policy:
132         train = policy.loc[policy.policy != held]
133         test = policy.loc[policy.policy == held].iloc[0]
134         fit = weighted_ols(train, columns)
135         x = np.array([1.0, *test[columns].to_numpy(float)])
136         pred = float(np.exp(x @ fit["beta"]))
137         rows.append({
138             "held_out_policy": held,
139             "observed_mean_eol": float(test.eol_mean),
140             "predicted_eol": pred,
141             "absolute_percentage_error": abs(pred - test.eol_mean) / test.
142                 eol_mean * 100,
143         })
144     return pd.DataFrame(rows)
145
146 def main() -> None:
147     OUT.mkdir(parents=True, exist_ok=True)
148     FIG.mkdir(parents=True, exist_ok=True)
149     summary = pd.read_csv(SUMMARY)
150     # 先在标签表中删除测试记录, 使其 EOL 不会进入后续 merge 的任何列。
151     audit = pd.read_csv(AUDIT)
152     audit = audit.loc[(audit.prediction_test == 0) & (audit.eol_status == "
153         available"),
154         ["battery_id", "matr_source_key", "external_eol_cycles"]]
155     data = summary.merge(audit, on="battery_id", how="inner", validate="
156         one_to_one")
157     data = data.loc[data.dataset_id == 3].copy()
158     data["policy"] = data.policy.map(canonical_policy)
159     # batch-2 的接续不是新电池; 主队列虽都是 batch-3, 仍保留这条保护。
160     data = data.sort_values("global_id").drop_duplicates("matr_source_key", keep

```

```

    ="first")
159 data = data.dropna(subset=["C1", "Q1", "C2", "external_eol_cycles"])
160 exposures = [exposure(r.C1, r.Q1, r.C2) for r in data.itertuples()]
161 data[["E_low", "E_high", "C_high"]] = exposures
162 data["log_eol"] = np.log(data.external_eol_cycles)
163
164 policy = data.groupby("policy", as_index=False).agg(
165     n=("battery_id", "size"), eol_mean=("external_eol_cycles", "mean"),
166     eol_median=("external_eol_cycles", "median"), eol_sd=(
167         "external_eol_cycles", "std"),
168     log_eol_mean=("log_eol", "mean"), C1=("C1", "mean"), Q1=("Q1", "mean"),
169     C2=("C2", "mean"), E_low=("E_low", "mean"), E_high=("E_high", "mean"),
170     C_high=("C_high", "mean"), battery_ids=("battery_id", lambda x: ", ".join(
171         map(str, sorted(x)))).sort_values("eol_mean", ascending=False)
172 kw = stats.kruskal(*[part.external_eol_cycles for _, part in data.groupby("
173     policy")])
174 kw_table = pd.DataFrame([
175     {"n_batteries": len(data), "n_policies": data.policy.nunique(), "H": kw.
176         statistic, "p_value": kw.pvalue,
177     "effect_epsilon_squared": max(0.0, (kw.statistic - data.policy.nunique()
178         + 1) / (len(data) - data.policy.nunique()))},
179 ])
180 pairs = holm_pairs(data)
181 coef, fit = fit_rows(policy, ["E_low", "E_high"], "two_window_log_eol_wls")
182 raw_coef, raw_fit = fit_rows(policy, ["C1", "Q1", "C2"], "
183     raw_parameter_log_eol_wls")
184 loo = leave_one_policy(policy, ["E_low", "E_high"])
185 no_short = policy.loc[policy.policy != "3_7C_31PER_5_9C"]
186 _, sensitivity = fit_rows(no_short, ["E_low", "E_high"], "drop_3_7C")
187 # 原始参数的相关性提示共线性; 只有 6 个设计点, 不能拆成独立边际效应。
188 corr = policy[["C1", "Q1", "C2", "E_low", "E_high"]].corr(method="spearman")
189
190 data.to_csv(OUT / "eol_cell_level.csv", index=False, encoding="utf-8-sig")
191 policy.to_csv(OUT / "eol_policy_summary.csv", index=False, encoding="utf-8-
192     sig")
193 kw_table.to_csv(OUT / "kruskal_eol.csv", index=False, encoding="utf-8-sig")
194 pairs.to_csv(OUT / "pairwise_eol_holm.csv", index=False, encoding="utf-8-sig
195     ")
196 pd.concat([coef, raw_coef], ignore_index=True).to_csv(OUT / "
197     wls_coefficients.csv", index=False, encoding="utf-8-sig")
198 pd.concat([fit, raw_fit, sensitivity], ignore_index=True).to_csv(OUT / "
199     wls_fit_summary.csv", index=False, encoding="utf-8-sig")
200 loo.to_csv(OUT / "leave_one_policy_eol.csv", index=False, encoding="utf-8-

```

```

    sig")
192 corr.to_csv(OUT / "spearman_design_correlation.csv", encoding="utf-8-sig")
193
194 plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans
    "]
195 plt.rcParams["axes.unicode_minus"] = False
196 order = list(policy.policy)
197 fig, ax = plt.subplots(figsize=(9, 5.2))
198 values = [data.loc[data.policy == name, "external_eol_cycles"] for name in
    order]
199 ax.boxplot(values, tick_labels=order, showmeans=True)
200 for pos, values_i in enumerate(values, start=1):
201     ax.scatter(np.full(len(values_i), pos), values_i, color="#2574a9", zorder
        =3)
202 ax.tick_params(axis="x", rotation=20, labelsz=8)
203 ax.set_ylabel("公开 EOL / 圈")
204 ax.set_title("同一 batch-3 结构下的策略寿命分布（非测试电池）")
205 fig.tight_layout()
206 fig.savefig(FIG / "policy_eol_boxplot.png", dpi=300)
207 plt.close(fig)
208 fig, ax = plt.subplots(figsize=(7.2, 5.0))
209 ax.scatter(policy.E_high, policy.eol_mean, s=50 + policy.n * 16, color="#
    d95f02")
210 for row in policy.itertuples():
211     ax.annotate(row.policy, (row.E_high, row.eol_mean), fontsize=7, xytext
        =(3, 4), textcoords="offset points")
212 ax.set_xlabel(r"高 SOC 电流暴露 $E^H$")
213 ax.set_ylabel("策略平均 EOL / 圈")
214 ax.set_title("高 SOC 暴露不能单独决定寿命")
215 fig.tight_layout()
216 fig.savefig(FIG / "eol_vs_high_soc_exposure.png", dpi=300)
217 plt.close(fig)
218
219 report = [
220     "# 问题二：以 EOL 为响应的寿命分析（脚本生成）", "",
221     "## 数据边界", "",
222     "- 仅纳入 `dataset_id=3`、`prediction_test=0` 且公开 EOL 可用的去重原电
        池。",
223     f"- 样本为 {len(data)} 块电池、{data.policy.nunique()} 个既有策略点。测试
        电池 EOL 标签在合并前已排除。",
224     "- 这是非正交、小样本的策略关联分析，不是 C1、Q1、C2 的独立因果实验。", "",
225     "## 策略汇总", "", markdown_table(policy), "",
226     "## 总体差异", "", markdown_table(kw_table, digits=5), "",
227     "Kruskal--Wallis 仅检验策略分布总体是否不同；两两比较需看 Holm 校正表。", "

```

```

228         ",
229         "## 两窗口寿命模型", "", markdown_table(fit, digits=5), "",
230         markdown_table(coef, digits=5), "",
231         "高 SOC 系数只能作为在六个策略点上的描述性关联。删除 3.7C--31%--5.9C 策略后
232         的拟合见 `wls_fit_summary.csv`, 用于检查结论是否依赖短寿策略。", "",
233         "## 留一策略外推", "", markdown_table(loo), "",
234         "留一策略误差说明该模型不应用于凭空设计远离已有配方的新策略。",
235     ]
236     (OUT / "report.md").write_text("\n".join(report), encoding="utf-8")
237     print(f"n={len(data)}, policies={data.policy.nunique()}, KW p={kw.pvalue:.6g
238           }")
239     print(policy[["policy", "n", "eol_mean", "eol_median"]].to_string(index=
240           False))
241     print(fit.to_string(index=False))
242     print(loo.to_string(index=False))
243
244 if __name__ == "__main__":
245     main()

```

B.4 问题三源程序

Listing 5 问题三 SOH 窗口验证与预测

```

1  """问题三小问 1 与 3: 不同早期窗口、策略特征的 SOH 预测比较。
2
3  训练及验证只使用赛题 CSV。每次以整块电池为单位留出，避免同一电池的循环点
4  同时出现在训练与验证中。外部 EOL 标签不被本脚本读取。
5  """
6
7  from __future__ import annotations
8
9  import json
10 import random
11 from pathlib import Path
12
13 import numpy as np
14 import pandas as pd
15 import torch
16 import matplotlib.pyplot as plt
17 from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
18 from sklearn.model_selection import KFold
19 from sklearn.preprocessing import StandardScaler
20 from torch import nn

```

```

21
22
23 ROOT = Path(__file__).resolve().parents[2]
24 OUT = ROOT / "A" / "问题三" / "output" / "window_ablation"
25 SEED = 20260815
26 HORIZON = 50
27 WINDOWS = (50, 100, 150)
28
29
30 def set_seed() -> None:
31     random.seed(SEED)
32     np.random.seed(SEED)
33     torch.manual_seed(SEED)
34     torch.use_deterministic_algorithms(True)
35
36
37 def make_features(frame: pd.DataFrame, meta: pd.Series, end: int, use_policy:
38     bool) -> np.ndarray:
39     """提取到 end 圈为止真正可观测的低维特征。"""
40     obs = frame[frame.cycle <= end].set_index("cycle").sort_index()
41     soh, ir, temp = obs.SOH_smooth, obs.IR, obs.Tavg
42     marks = np.linspace(1, end, 7).round().astype(int)
43     first, split = soh.loc[1], max(2, end // 2)
44     dynamic = [
45         *[float(soh.loc[m] - first) for m in marks],
46         float((soh.loc[end] - soh.loc[split]) / (end - split)),
47         float((soh.loc[split] - first) / (split - 1)),
48         float(ir.loc[split:end].mean() - ir.loc[1:split].mean()),
49         float(temp.loc[split:end].mean() - temp.loc[1:split].mean()),
50     ]
51     if not use_policy:
52         return np.asarray(dynamic, dtype=np.float32)
53     c1 = 0.0 if pd.isna(meta.C1) else float(meta.C1)
54     return np.asarray([c1, float(meta.Q1), float(meta.C2), *dynamic], dtype=np.
55         float32)
56
57
58 def make_target(frame: pd.DataFrame, end: int) -> np.ndarray:
59     soh = frame.set_index("cycle").SOH_smooth
60     return (soh.loc[np.arange(end + 1, end + HORIZON + 1)].to_numpy() - soh.loc[
61         end]).astype(np.float32)
62
63
64 class SmallMLP(nn.Module):

```

```

62     def __init__(self, n_input: int) -> None:
63         super().__init__()
64         self.net = nn.Sequential(nn.Linear(n_input, 16), nn.ReLU(), nn.Linear(16,
65                                     8), nn.ReLU(), nn.Linear(8, HORIZON))
66
67     def forward(self, x: torch.Tensor) -> torch.Tensor:
68         return self.net(x)
69
70 def mlp_predict(x_fit: np.ndarray, y_fit: np.ndarray, x_eval: np.ndarray) -> np
71     .ndarray:
72     """在每个训练折内部独立标准化；固定训练轮数以免用验证折早停。"""
73     sx, sy = StandardScaler(), StandardScaler()
74     xf = torch.tensor(sx.fit_transform(x_fit), dtype=torch.float32)
75     yf = torch.tensor(sy.fit_transform(y_fit), dtype=torch.float32)
76     xe = torch.tensor(sx.transform(x_eval), dtype=torch.float32)
77     model = SmallMLP(x_fit.shape[1])
78     opt = torch.optim.AdamW(model.parameters(), lr=0.006, weight_decay=0.03)
79     for _ in range(700):
80         opt.zero_grad()
81         loss = nn.functional.mse_loss(model(xf), yf)
82         loss.backward()
83         opt.step()
84     with torch.no_grad():
85         return sy.inverse_transform(model(xe).numpy())
86
87 def enforce_monotone(delta: np.ndarray) -> np.ndarray:
88     return np.minimum.accumulate(np.minimum(delta, 0.0), axis=1)
89
90
91 def linear_predict(frame: pd.DataFrame, end: int) -> np.ndarray:
92     tail = frame[(frame.cycle > end - min(50, end - 1)) & (frame.cycle <= end)]
93     return np.polyval(np.polyfit(tail.cycle, tail.SOH_smooth, 1), np.arange(end
94                                     + 1, end + HORIZON + 1))
95
96 def main() -> None:
97     set_seed()
98     OUT.mkdir(parents=True, exist_ok=True)
99     summary = pd.read_csv(ROOT / "data" / "battery_summary.csv").set_index("
100         battery_id")
101     cycles = pd.read_csv(ROOT / "data" / "cycle_train.csv")
102     full_ids = summary.index[(summary.prediction_test.eq(0)) & (cycles.groupby("

```

```

        battery_id").cycle.max().eq(200))].to_numpy()
102 rows: list[dict[str, object]] = []
103 for end in WINDOWS:
104     for use_policy in (False, True):
105         x = np.vstack([make_features(cycles[cycles.battery_id == bid],
106                                 summary.loc[bid], end, use_policy) for bid in full_ids])
107         y = np.vstack([make_target(cycles[cycles.battery_id == bid], end) for
108                             bid in full_ids])
109         for fold, (fit, valid) in enumerate(KFold(5, shuffle=True,
110                                     random_state=SEED).split(full_ids), 1):
111             set_seed()
112             delta = enforce_monotone(mlp_predict(x[fit], y[fit], x[valid]))
113             for pos, bid in enumerate(full_ids[valid]):
114                 frame = cycles[cycles.battery_id == bid]
115                 base = frame.loc[frame.cycle.eq(end), "SOH_smooth"].iloc[0]
116                 actual = base + y[valid][pos]
117                 mlp = base + delta[pos]
118                 linear = linear_predict(frame, end)
119                 for method, prediction in (("torch_mlp", mlp), ("linear_tail",
120                                     linear)):
121                     rows.append({
122                         "window_end": end, "use_policy": use_policy, "fold":
123                             fold, "battery_id": int(bid), "method": method,
124                         "rmse": float(np.sqrt(mean_squared_error(actual,
125                             prediction))), "mae": float(mean_absolute_error(
126                             actual, prediction)),
127                         "r2": float(r2_score(actual, prediction)),
128                     })
129 detail = pd.DataFrame(rows).sort_values(["window_end", "use_policy", "method",
130     "battery_id"])
131 detail.to_csv(OUT / "cv_detail.csv", index=False, encoding="utf-8-sig")
132 report = detail.groupby(["window_end", "use_policy", "method"], as_index=
133     False).agg(
134     batteries=("battery_id", "count"), rmse_mean=("rmse", "mean"), mae_mean=(
135         "mae", "mean"), r2_mean=("r2", "mean")
136 )
137 report.to_csv(OUT / "cv_summary.csv", index=False, encoding="utf-8-sig")
138 # 对题设 9 块测试电池, 采用在 end=150 上验证最优的线性尾段外推。
139 test_ids = summary.index[summary.prediction_test.eq(1)].to_numpy()
140 prediction_rows: list[dict[str, object]] = []
141 for bid in test_ids:
142     forecast = linear_predict(cycles[cycles.battery_id == bid], 150)
143     prediction_rows.extend(
144         {"battery_id": int(bid), "cycle": int(cycle), "SOH_prediction": float

```



```

        (value), "method": "linear_tail_50_selected"}
135     for cycle, value in zip(np.arange(151, 201), forecast)
136     )
137 final_prediction = pd.DataFrame(prediction_rows)
138 final_prediction.to_csv(OUT / "selected_test_soh_151_200.csv", index=False,
    encoding="utf-8-sig")
139 final_prediction.groupby("battery_id", as_index=False).agg(soh_151=(
    SOH_prediction", "first"), soh_200=("SOH_prediction", "last")).merge(
140     summary[["policy"]], left_on="battery_id", right_index=True
141 ).to_csv(OUT / "selected_test_soh_overview.csv", index=False, encoding="utf
    -8-sig")
142 plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans
    "]
143 plt.rcParams["axes.unicode_minus"] = False
144 fig, ax = plt.subplots(figsize=(10, 5.4))
145 for bid in test_ids:
146     observed = cycles[cycles.battery_id.eq(bid)]
147     predicted = final_prediction[final_prediction.battery_id.eq(bid)]
148     color = ax.plot(observed.cycle, observed.SOH_smooth, linewidth=1.15,
        label=f"{bid}: 观测")[0].get_color()
149     ax.plot(predicted.cycle, predicted.SOH_prediction, "--", color=color,
        linewidth=1.7)
150 ax.axvline(150, color="black", linestyle=":", linewidth=1, label="第150圈:
    预测起点")
151 ax.set(xlabel="循环次数", ylabel="平滑 SOH", title="问题三：测试电池第151
    --200圈 SOH 预测")
152 ax.legend(ncol=2, fontsize=8)
153 fig.tight_layout()
154 fig.savefig(OUT / "selected_test_soh_forecast.png", dpi=220)
155 plt.close(fig)
156 (OUT / "README.md").write_text(
157     "# 问题三：窗口与策略特征消融\n\n"
158     "每一行验证结果都由一块完整电池的连续 50 圈构成；整块电池不会出现在对应训练
        折。"
159     "`use_policy=True` 表示加入 C1、Q1、C2；外部 EOL 标签没有被读取。\\n",
160     encoding="utf-8",
161 )
162 print(report.to_string(index=False))
163 print(json.dumps({"full_batteries": int(len(full_ids)), "windows": list(
    WINDOWS), "horizon": HORIZON}, ensure_ascii=False))
164
165
166 if __name__ == "__main__":
167     main()

```

Listing 6 问题三隔离 EOL 外部标定

```

1  """问题三：以公开 MATR 非测试 EOL 进行隔离标定。
2
3  本文件的边界非常重要：MATR_labels.json 是原始公开数据的寿命标签，
4  它不属于赛题附件。本脚本只将 prediction_test=0 的记录用于拟合和模型选择；
5  对 prediction_test=1 的九块电池，绝不读取其 external_eol_cycles 列。
6  因此输出是“带公开外部先验的寿命估计”，而不是仅靠附件可直接验证的答案。
7  """
8
9  from __future__ import annotations
10
11  from pathlib import Path
12
13  import numpy as np
14  import pandas as pd
15  from sklearn.ensemble import ExtraTreesRegressor, RandomForestRegressor
16  from sklearn.impute import SimpleImputer
17  from sklearn.linear_model import Ridge
18  from sklearn.metrics import mean_absolute_error, mean_squared_error
19  from sklearn.model_selection import LeaveOneOut, cross_val_predict
20  from sklearn.pipeline import make_pipeline
21  from sklearn.preprocessing import StandardScaler
22
23
24  ROOT = Path(__file__).resolve().parents[2]
25  SUMMARY_PATH = ROOT / "data" / "battery_summary.csv"
26  CYCLES_PATH = ROOT / "data" / "cycle_train.csv"
27  AUDIT_PATH = ROOT / "references" / "matr_source" / "external_eol_audit" / "
    contest_to_matr_eol_audit.csv"
28  OUT_DIR = ROOT / "A" / "问题三" / "output" / "external_eol_calibration"
29
30
31  def slope(x: np.ndarray, y: np.ndarray) -> float:
32      return float(np.polyfit(x, y, 1)[0]) if len(x) >= 2 else np.nan
33
34
35  def feature_row(summary: pd.Series, cell: pd.DataFrame) -> dict[str, float]:
36      """从截至第 150 圈的赛题可见信息提取少量、可解释的特征。"""
37      cell = cell.sort_values("cycle").query("cycle <= 150")
38      cyc = cell["cycle"].to_numpy(dtype=float)
39      soh = cell["SOH_smooth"].to_numpy(dtype=float)
40      row: dict[str, float] = {
41          "battery_id": int(summary.battery_id),
42          "C1": float(summary.C1) if pd.notna(summary.C1) else np.nan,

```

```

43     "Q1": float(summary.Q1),
44     "C2": float(summary.C2),
45     "initial_capacity": float(summary.initial_capacity),
46     "mean_chargetime": float(summary.mean_chargetime),
47     "mean_IR": float(summary.mean_IR),
48     "mean_Tavg": float(summary.mean_Tavg),
49 }
50 for point in (1, 25, 50, 75, 100, 125, 150):
51     nearest = int(np.argmin(np.abs(cyc - point)))
52     row[f"soh_{point}"] = float(soh[nearest])
53 for left, right in ((1, 50), (51, 100), (101, 150), (1, 150)):
54     mask = (cyc >= left) & (cyc <= right)
55     row[f"slope_{left}_{right}"] = slope(cyc[mask], soh[mask])
56 for col in ("IR", "Tavg", "chargetime"):
57     values = cell[col].to_numpy(dtype=float)
58     row[f"{col}_late_minus_early"] = float(np.nanmean(values[-25:]) - np.
        nanmean(values[:25]))
59 return row
60
61
62 def metrics(y: np.ndarray, pred_log: np.ndarray) -> dict[str, float]:
63     pred = np.exp(pred_log)
64     return {
65         "rmse_cycles": float(mean_squared_error(y, pred) ** 0.5),
66         "mae_cycles": float(mean_absolute_error(y, pred)),
67         "median_ape_pct": float(np.median(np.abs(pred - y) / y) * 100),
68     }
69
70
71 def main() -> None:
72     OUT_DIR.mkdir(parents=True, exist_ok=True)
73     summary = pd.read_csv(SUMMARY_PATH)
74     cycles = pd.read_csv(CYCLES_PATH)
75     audit = pd.read_csv(AUDIT_PATH)
76     merged = summary.merge(
77         audit[["battery_id", "matr_source_key", "external_eol_cycles", "
            eol_status"]],
78         on="battery_id", how="left", validate="one_to_one",
79     )
80     # 审计文件可能保存了测试对象的公开标签; 从这一行起完全脱敏, 避免标签进入
81     # 特征表、输出目录或任何预测调用。matr_source_key 保留, 仅用于识别接续记录。
82     test_mask = merged["prediction_test"].eq(1)
83     merged.loc[test_mask, "external_eol_cycles"] = pd.NA
84     merged.loc[test_mask, "eol_status"] = "withheld_test_label"

```

```

85
86 records = []
87 # 先对每个赛题记录提取特征；测试集中的续跑记录仍需各自给出预测。
88 for row in merged.sort_values("global_id").itertuples(index=False):
89     cell = cycles.loc[cycles.battery_id == row.battery_id]
90     item = feature_row(row, cell)
91     item.update({
92         "global_id": int(row.global_id),
93         "prediction_test": int(row.prediction_test),
94         "matr_source_key": row.matr_source_key,
95         "external_eol_cycles": row.external_eol_cycles,
96         "eol_status": row.eol_status,
97     })
98     records.append(item)
99 frame = pd.DataFrame(records)
100 # 仅训练/验证时去重：batch-2 的续跑不是新的寿命样本。
101 train = (frame.query("prediction_test == 0 and eol_status == 'available'")
102         .sort_values("global_id").drop_duplicates("matr_source_key", keep="
            first").copy())
103 test = frame.query("prediction_test == 1").copy()
104 assert train.external_eol_cycles.notna().all()
105
106 feature_cols = [c for c in frame.columns if c not in {
107     "battery_id", "global_id", "prediction_test", "matr_source_key",
108     "external_eol_cycles", "eol_status",
109 }]
110 x_train = train[feature_cols]
111 y_train = train.external_eol_cycles.to_numpy(dtype=float)
112 x_test = test[feature_cols]
113 # 对数寿命会让 500 与 2000 圈的相对误差获得较公平的权重。
114 log_y = np.log(y_train)
115 candidates = {
116     "ridge_log_eol": make_pipeline(SimpleImputer(strategy="median"),
117                                   StandardScaler(), Ridge(alpha=10.0)),
117     "random_forest_log_eol": make_pipeline(SimpleImputer(strategy="median"),
118                                           RandomForestRegressor(
119       n_estimators=500, min_samples_leaf=2, max_features=0.7, random_state
120         =20260814)),
119     "extra_trees_log_eol": make_pipeline(SimpleImputer(strategy="median"),
120                                           ExtraTreesRegressor(
121       n_estimators=500, min_samples_leaf=2, max_features=0.8, random_state
122         =20260814)),
121 }
122 loo = LeaveOneOut()

```

```

123 summary_rows = []
124 oof = pd.DataFrame({"battery_id": train.battery_id, "true_eol_cycles":
    y_train})
125 for name, model in candidates.items():
126     pred_log = cross_val_predict(model, x_train, log_y, cv=loo, n_jobs=-1)
127     row = {"model": name, "samples": len(train), **metrics(y_train, pred_log)}
128     summary_rows.append(row)
129     oof[f"{name}_oof_eol"] = np.exp(pred_log)
130 score = pd.DataFrame(summary_rows).sort_values("median_ape_pct")
131 selected_name = str(score.iloc[0].model)
132 selected = candidates[selected_name].fit(x_train, log_y)
133 # 这是唯一用于测试对象的 fit/predict 调用：没有访问其 external_eol_cycles。
134 test_pred_log = selected.predict(x_test)
135 prediction = test[["battery_id", "global_id", "matr_source_key"]].copy()
136 prediction["record_level_eol_prediction_for_diagnosis"] = np.exp(
    test_pred_log)
137 # 同一原始电池的续跑记录对应同一个 EOL；用对数均值合并，等价于几何平均。
138 prediction["estimated_eol_cycles_external_calibration"] = np.exp(
139     prediction.groupby("matr_source_key")["
        record_level_eol_prediction_for_diagnosis"].transform(
140         lambda values: np.log(values).mean()
141     )
142 )
143 prediction["duplicate_source_resolution"] = "
    geometric_mean_within_same_matr_source_key"
144 prediction["model"] = selected_name
145 prediction["data_boundary"] = "
    trained_only_on_unique_non_test_external_labels"
146
147 frame.to_csv(OUT_DIR / "feature_table.csv", index=False, encoding="utf-8-sig")
148 score.to_csv(OUT_DIR / "loo_model_comparison.csv", index=False, encoding="
    utf-8-sig")
149 oof.to_csv(OUT_DIR / "loo_predictions_non_test_only.csv", index=False,
    encoding="utf-8-sig")
150 prediction.to_csv(OUT_DIR / "estimated_test_eol.csv", index=False, encoding=
    "utf-8-sig")
151 readme = f"""# 外部 EOL 标定（问题三）
152
153 - 可用于训练和留一验证的样本：{len(train)} 个去重的非测试原电池。
154 - 9 块测试电池在读取后即从标签列排除；输出预测时未访问其 EOL。
155 - 模型按留一交叉验证的中位相对误差选择：`{selected_name}`。
156 - 这是一项公开 MATR 原始数据标签的外部标定审计，不能与“只用赛题附件”的短期 SOH 验

```

```

    证混为一谈。
157 """
158 (OUT_DIR / "README.md").write_text(readme, encoding="utf-8")
159 print(score.to_string(index=False))
160 print("selected:", selected_name)
161 print(prediction.to_string(index=False))
162
163
164 if __name__ == "__main__":
165     main()

```

C AI 工具使用声明

根据赛事论文格式规范，现就人工智能工具使用情况作如下声明。全文不出现参赛者姓名与学校信息。

C.1 所用工具

- OpenAI ChatGPT（记录版本：1.2026.190，模型 gpt5.6-sol-high）
- OpenAI Codex CLI（记录版本：v0.147.0，模型 gpt5.6-sol-high；DeepSeek-V4-Flash-0731）
- 对话式编程助手（用于文献整理、问题一结果写入 LaTeX 论文框架）

C.2 使用目的与环节

1. 讨论与协作：询问 Git 多人协作与仓库管理相关操作说明。
2. 问题一：依据人工撰写的 AGENTS.md 生成模块化分析代码，完成 IQR 清洗、策略统一（9 种合并为 7 种）、200 圈汇总、49 张单体曲线与总图；人工检查可运行性后调整参数并复现 log₃。
3. 文献：整理快充、SOH 预测与策略优化相关公开文献条目，写入 references/。
4. 论文：将第三次迭代后的问题一成果写入 LaTeX 正文；问题二统计分析结果由队友写入相应章节后，再与问题一典型策略口径对齐。

C.3 提示方式

主要方式为：先由人工完成题意分析与方法构想，再将任务写成 AGENTS.md 或结构化提示，交由 AI 生成代码或文档草稿，随后人工运行、核验数值与图件是否与数据一致。问题一提示词原文见 A/问题一/AGENTS.md 及 log₂.md 附录，此处不重复粘贴以免附录过长；支撑材料中保留全文。

C.4 采纳、修改与核验

- 问题一流水线经 `log_3` 与验收确认：IQR 替换 559 格、策略 9→7、200 圈样本 40 块、分位阈值 0.993433/0.996103、49 张单体曲线与 `SOH_ALL` 齐全，与正文一致。
- 典型长/短寿命以 A/问题一/总结.md 为准：不以 `policy.csv` 中按中位数标记的 4.8C 组合并为“寿命最长”代表；因果表述保持为初步观察。
- 问题二已写入 Kruskal–Wallis、回归诊断与混合效应结果；问题三、四仍为框架。
AI 仅作为辅助工具，论文观点、模型选择与最终结论由参赛队负责。